Comentarios del código · Adenda

SOBUSA Data Challenge 2026 · Frecuencias inteligentes

7 de noviembre de 2026

El código del proyecto sigue sin comentarios ni docstrings; `tests/test_limpieza.py` lo verifica. Esta adenda recoge lo que cambió después del 29 de octubre de 2026. El documento de esa fecha sigue sin cambios desde la página siguiente a esta adenda. Donde ambos hablen del mismo archivo, prevalece la adenda.

Registro de cambios

- **Perfil por segmento.** Con el usuario restringido, el ciclo marcaba como fallida la actualización de segmentos: `recalcular_perfil` borra el perfil anterior y el rol de aplicación no tenía `DELETE`. El ciclo toleraba el error y terminaba como exitoso, así que el perfil se quedó sin actualizar.
- **Arranque sin datos ni modelos.** Desde un clon limpio la API se reiniciaba en bucle. Ahora arranca, y `/salud` explica qué falta. Ya no presenta datos sintéticos como si fueran reales.
- **Fechas con zona.** Las respuestas de la API publican la zona, como pide §12.1.
- **Recaudo.** Cada recomendación estima los pasajeros esperados y en riesgo de no abordar; con `tarifa_pasajero` los expresa en pesos.
- **Recarga por rango.** Es el pendiente de E02: relee un periodo corregido sin mover la marca de agua.
- **Reproducibilidad.** El paquete de demostración deja el sistema operando desde un clon limpio, sin acceso a la base del operador.
- **Cifras de redistribución.** Las de E01, E02 y E12 se recalcularon con un script reproducible y con los modelos vigentes antes del periodo evaluado (`docs/evidencias/redistribucion.json`).

Notas por archivo

Archivo	Estado	Nota
`migrations/versions/0010_permiso_perfil_segmentos.py`	nuevo	`DELETE` solo sobre `segment_profile`, un resumen derivado que cada ciclo reemplaza. La historia sigue sin poder borrarse.
`migrations/versions/0011_tarifa_pasajero.py`	nuevo	Parámetro sin valor hasta que el operador lo entregue.
`frecuencias/segmentos/carga.py`	modificado	`refrescar_perfil` y `--solo-perfil` recalculan con las observaciones guardadas, sin leer la base del operador.
`frecuencias/inferencia/modelos.py`	modificado	`SinModelos` distingue la falta de modelos de un artefacto alterado, que sí debe detener el arranque.
`frecuencias/api/main.py`	modificado	`SinHistorico` y `_preparar` dejan arrancar la API; `_vista` responde `503` con el motivo; `_limpio` agrega la zona.
`frecuencias/api/esquemas.py`	modificado	`FechaConZona` localiza las fechas de los esquemas en America/Bogota.
`frecuencias/optimizacion/frecuencia.py`	modificado	`estimar_recaudo` usa el exceso de carga sobre la capacidad nominal en el tramo más cargado. No entra en la función objetivo.

Archivo	Estado	Nota
`frecuencias/optimizacion/parametros.py`	modificado	Agrega `tarifa` a `ParametrosHora`.
`frecuencias/operacion/recomendacion.py`	modificado	Guarda el recaudo en `parametros`.
`frecuencias/ingesta/historico.py`	modificado	`leer_archivos`, `_guardar_carga` y `recargar_rango`. Una recarga sin filas no calcula marca de agua.
`frecuencias/operacion/ciclo.py`	modificado	Usa `historico.leer_archivos` en lugar de su propia lectura.
`frecuencias/tablero/app.js`, `index.html`	modificado	Aviso de sistema sin preparar y datos de recaudo en la tarjeta.
`tests/test_contrato.py`	nuevo	Compara la API con `docs/openapi.json`, comprueba los endpoints de §12.2 y el error uniforme, y corre en CI.
`tests/test_historico.py`	nuevo	Rango, recarga, fallos y marca de agua.
`tests/test_segmentos.py`, `test_roles.py`	modificado	Recalculan el perfil con el rol restringido. Sin el permiso, la prueba falla con el mismo error de la base.
`pruebas/paquete_demo.py`	nuevo	Exporta e importa datos y modelos. Se niega a restaurar sobre una base con datos sin `--reemplazar`.
`pruebas/demo_ciclo.py`	nuevo	Corre el ciclo completo de §14.2 por la API y guarda las decisiones. Solo usa la biblioteca estándar.
`pruebas/sesgo_segmentos.py`	nuevo	Mide el reparto por segmento con cuotas calculadas solo con datos previos a la ventana.
`pruebas/informe_modelos.py`	nuevo	Resume los modelos activos desde el registro.
`pruebas/redistribucion.py`	nuevo	Simula el ciclo hora a hora sin escribir en la base: frecuencia recomendada frente a la operada, línea base y vehículo-horas.

Verificación

La suite completa pasa con una copia de la base: 412 pruebas y 80 % de cobertura. En modo CI, con base vacía, pasan 394, se omite 1 y 17 quedan fuera por requerir datos y modelos reales. Desde una copia limpia, el sistema llegó a un ciclo completo sin la base del operador (`docs/evidencias/clon_limpio.json`), y el tablero se aprobó en Microsoft Edge.

Comentarios del código

SOBUSA Data Challenge 2026 · Frecuencias inteligentes · actualizado el 29 de octubre de 2026

El código del proyecto sigue sin comentarios ni docstrings; una prueba automática (tests/test_limpieza.py) lo verifica. Este documento reúne la explicación de lo hecho en las fases 2 a 8 de la especificación y en el cierre de pendientes del 29 de octubre: primero el registro de cambios, la verificación y los resultados, después las notas de cada archivo nuevo o modificado, con la línea del código actual a la que acompañan, la función donde están, la fecha y, en gris, esa línea. Al final, como anexo, va sin cambios el documento de la fase 1 (15 de octubre de 2026) con su propia paginación. Donde un archivo se reescribió en esta fase (frecuencias/api/main.py, esquemas.py, modelos.py y entrenamiento.py), las notas de este documento reemplazan a las del anexo.

Archivo	Estado	Líneas	Notas
frecuencias/config.py	modificado	24	1
frecuencias/tiempo.py	nuevo	19	3
frecuencias/observabilidad.py	nuevo	93	7
frecuencias/seguridad.py	nuevo	87	5
frecuencias/resiliencia.py	nuevo	72	2
frecuencias/ingesta/consultas.py	modificado	65	5
frecuencias/ingesta/historico.py	nuevo	211	7
frecuencias/analitica/calidad.py	reescrito	134	6
frecuencias/segmentos/carga.py	nuevo	296	9
frecuencias/segmentos/consultas.py	nuevo	85	3
frecuencias/persistencia/tablas.py	nuevo	333	6
frecuencias/persistencia/consultas.py	nuevo	20	2
frecuencias/persistencia/parametros.py	nuevo	55	2
frecuencias/persistencia/ejecuciones.py	nuevo	51	4
frecuencias/persistencia/registro_modelos.py	nuevo	65	2
frecuencias/persistencia/predicciones.py	nuevo	52	2
frecuencias/persistencia/recomendaciones.py	nuevo	31	0
frecuencias/persistencia/alertas.py	nuevo	123	4
frecuencias/persistencia/calidad.py	nuevo	35	1
frecuencias/inferencia/evaluacion.py	modificado	74	5
frecuencias/inferencia/ajuste.py	nuevo	66	4
frecuencias/inferencia/cuantiles.py	nuevo	53	4
frecuencias/inferencia/diagnostico.py	nuevo	40	3
frecuencias/inferencia/artefactos.py	nuevo	97	5
frecuencias/inferencia/prediccion.py	nuevo	107	5
frecuencias/inferencia/modelos.py	reescrito	36	2
frecuencias/inferencia/entrenamiento.py	reescrito	212	6
frecuencias/inferencia/retencion.py	nuevo	90	4
frecuencias/optimizacion/parametros.py	modificado	92	4
frecuencias/optimizacion/frecuencia.py	nuevo	70	6
frecuencias/optimizacion/oferta.py	nuevo	32	3
frecuencias/alertas/reglas.py	modificado	180	9
frecuencias/alertas/notificaciones.py	reescrito	142	5
frecuencias/operacion/pronostico.py	nuevo	47	2
frecuencias/operacion/recomendacion.py	modificado	70	4
frecuencias/operacion/servicio.py	nuevo	50	3
frecuencias/operacion/reentrenamiento.py	nuevo	84	4
frecuencias/operacion/ciclo.py	modificado	207	11
frecuencias/operacion/programador.py	nuevo	44	2
frecuencias/api/main.py	reescrito	608	14
frecuencias/api/esquemas.py	reescrito	102	4
frecuencias/api/errores.py	modificado	45	2
frecuencias/tablero/index.html	modificado	150	4
frecuencias/tablero/app.js	modificado	452	7
frecuencias/tablero/estilos.css	modificado	152	0
migrations/versions/0002_historico_calidad_notificaciones.py	nuevo	150	2
migrations/versions/0003_parametros_operacion.py	nuevo	127	2
migrations/versions/0004_calibracion_alertas.py	nuevo	37	1
migrations/versions/0005_claves_api.py	nuevo	36	2
migrations/versions/0006_segmentos.py	nuevo	93	3
tests/test_seguridad.py	nuevo	53	0
tests/test_resiliencia.py	nuevo	77	0
tests/test_notificaciones.py	nuevo	100	0
tests/test_retencion.py	nuevo	90	0
tests/test_segmentos.py	nuevo	82	0
tests/test_ciclo.py	nuevo	50	0
tests/test_reentrenamiento.py	nuevo	24	0
tests/test_optimizacion.py	nuevo	127	0
tests/test_alertas.py	nuevo	113	0
tests/test_artefactos.py	nuevo	134	0

Archivo	Estado	Líneas	Notas
tests/test_calidad.py	nuevo	68	0
tests/test_limpieza.py	nuevo	34	0
tests/test_api.py	reescrito	151	0
pruebas/e2e_tablero.py	nuevo	168	0
pruebas/carga_api.py	nuevo	102	0
pruebas/volumen_ingesta.py	nuevo	93	0
pruebas/explain_fuente.py	nuevo	63	1
pruebas/humo_docker.py	nuevo	70	0
pruebas/informe_sonar.py	nuevo	81	0
Dockerfile	modificado	28	1
docker-compose.yml	modificado	90	4
.env.example	modificado	40	0
requirements.txt	modificado	20	0
requirements-dev.txt	modificado	7	0
.github/workflows/ci.yml	modificado	62	3
sonar-project.properties	modificado	10	1
.coveragerc	nuevo	6	0
pytest.ini	modificado	9	0
Total		7548	213

Registro de cambios

29 de octubre de 2026 · Cierre de pendientes

- Calibración de alertas con umbrales versionados en PostgreSQL (migración 0004): ALT-001 con la condición del §5.2 y tolerancia de 2 buses o 15 % para ALT-003; resumen de alertas con porcentaje de descartadas (§9.4).
- Seguridad de la API: claves por rol (lector, operador, administrador) guardadas como huella SHA-256 (migración 0005), auditoría con el nombre de la clave y Swagger restringido por ambiente (§12.1).
- Notificaciones reales por SMTP (TLS obligatorio con certificado verificado) y WhatsApp Cloud API, con timeout, reintento y circuit breaker (RNF-010); sin credenciales se siguen simulando. La lectura de la base del reto usa el mismo mecanismo, y si no responde se genera ALT-009 y la marca de agua no avanza.
- Errores saneados: ningún mensaje guardado o registrado contiene el host, la IP ni el usuario de la base.
- Segmentos (migración 0006): la lógica de los scripts del operador se reproduce sin tocar su base, validada en 99,7 %; perfil por segmento con promedios, mínimos y máximos (§6.1), carga a bordo balanceada, tramo crítico en cada recomendación y predicción desagregada por segmento. Se actualiza en el ciclo con origen base.
- Retención de modelos: archiva rechazados y retirados antiguos sin borrar y conserva lo necesario para revertir.
- Correcciones: hora de fin de las ejecuciones con el reloj de la base (fallo intermitente), detalle de las ejecuciones combinado en vez de reemplazado, ALT-010 solo con 3 días observados y solo del modelo activo, intervalo no calibrado y sin recomendación fuera del servicio habitual, keep-alive de 30 s en uvicorn y consulta de conteo 6 veces más rápida.
- Verificación completa: ciclo contra la base del reto, Docker Compose con ciclo asíncrono y programador, tablero en Edge, carga, volumen, EXPLAIN y SonarQube sin hallazgos (detalle en la sección siguiente).
- Artículo actualizado a LightGBM y cuaderno de variables apuntando a los modelos archivados de la fase 1.

28 de octubre de 2026 · Fases 2 a 8

- Modelo de producción: LightGBM con objetivo de Poisson. En la comparación a 60 min fue el de menor WAPE y desviación de Poisson entre seis variantes de gradient boosting (tabla más abajo) y entrena en menos de la mitad del tiempo que XGBoost. XGBoost se sigue entrenando en cada ejecución solo como referencia.

- Ajuste de hiperparámetros con Optuna (TPE, semilla fija) y validación temporal walk-forward de 4 pliegues, minimizando la desviación de Poisson. La configuración base se evalúa primero, así el ajuste nunca empeora.
- Intervalo de confianza del 80 % (RF-006) con tres modelos cuantílicos calibrados por predicción conforme (CQR), comparado con cuantiles sin calibrar y con un intervalo de ancho constante.
- Diagnóstico guardado en el reporte: importancia por ganancia y por SHAP (contribuciones de LightGBM), pruebas de Ljung-Box y McLeod-Li sobre los residuos, d de Cohen para el sobreajuste y error por ruta, tipo de día y hora pico o valle.
- Paquete de modelo con manifiesto SHA-256, registro campeón por horizonte en model_registry y verificación de integridad al cargar o activar.
- Predicción en servicio a 15, 30, … 120 min con el menor horizonte que cubre cada antelación, explicación de las cinco variables más influyentes y guardado idempotente con instantánea de variables.
- Optimización de frecuencia (Fase 3) con parámetros versionados por ruta, franja y tipo de día medidos en el análisis de flota 2025 (migración 0003), escenarios de sensibilidad y viabilidad por headway mínimo y flota.
- Alertas ALT-001 a ALT-010 (Fase 4) con deduplicación, ciclo de vida con historial y notificaciones simuladas por correo y WhatsApp.
- Calidad de datos como catálogo CAL-001 a CAL-104 con dimensión, severidad y tratamiento; cada problema queda guardado por ejecución.
- Histórico propio en ai_frecuencias (migración 0002) con carga COPY + upsert, extracción incremental con marca de agua y ciclo completo con reentrenamiento campeón/retador (Fase 5); programador con APScheduler.
- Observabilidad (logs JSON con correlation_id, métricas Prometheus, errores uniformes), CI con migraciones de ida y vuelta, cobertura, SonarQube opcional y construcción de la imagen (Fase 6).
- API v1 completa según el §12.2 y tablero web accesible con exportación a CSV y Excel (Fase 7).
- Correcciones: un X-Correlation-Id que no es UUID ya no rompe el registro de la ejecución; la oferta actual convierte las fechas aunque no haya viajes; los viajes leídos del parquet convierten cancelada S/N a booleano.
- Corrección encontrada en la demostración: el campeón/retador podía comparar al activo contra una copia de sí mismo (ventana que empezaba en el corte del activo) y un empate activaba el retador. Ahora la ventana deja días nuevos al retador y solo una mejora lo activa. Los tres modelos activados por ese empate se marcaron como rechazados y se reactivaron los del entrenamiento completo. Además, la ejecución de ingesta conserva el enlace con su ciclo.

Verificación

Verificación al cierre del 29 de octubre: 224 pruebas, todas en verde (incluidas las 11 lentas de la API con la base, los datos y los modelos reales), con 74 % de cobertura de líneas del paquete frecuencias. La imagen de Docker se construye con las dependencias nuevas y la API queda sana en Docker Compose. El entrenamiento completo de los tres horizontes, con 15 trials de Optuna cada uno, tardó 37 minutos.

Comandos

Tarea	Comando
Crear o actualizar ai_frecuencias	`python -m alembic upgrade head`
Crear una clave de acceso	`python -m frecuencias.seguridad crear --nombre ana --rol operador`
Cargar el histórico inicial	`python -m frecuencias.ingesta.historico --hasta 2025-12-01`
Cargar segmentos de un periodo	`python -m frecuencias.segmentos.carga --desde 2025-11-03 --hasta 2025-12-01`
Validar segmentos contra el operador	`python -m frecuencias.segmentos.carga --validar 2023-01-17`

Tarea	Comando
Entrenar, registrar y activar	`python -m frecuencias.inferencia.entrenamiento --trials 15 --hasta 2025-12-01`
Archivar modelos antiguos	`python -m frecuencias.inferencia.retencion --simular`
Correr el ciclo incremental	`python -m frecuencias.operacion.ciclo --origen base --dias 1 --reentrenar auto`
Simular días con datos locales	`python -m frecuencias.operacion.ciclo --origen archivos --dias 3`
Programar el ciclo	`python -m frecuencias.operacion.programador --cron "15 * * * *"`
API y tablero en el equipo	`uvicorn frecuencias.api.main:app` (tablero en `/tablero`)
Todo con Docker	`docker compose up -d --build`
Con el programador	`docker compose --profile programador up -d`
Pruebas rápidas	`python -m pytest -m "not lenta"`
Todas las pruebas	`python -m pytest` (requiere base, datos y modelos)
Tablero en el navegador	`python pruebas/e2e_tablero.py --url http://localhost:8000`
Carga, volumen y EXPLAIN	`python pruebas/carga_api.py` · `python pruebas/volumen_ingesta.py` · `python pruebas/explain_fuente.py`
SonarQube local	`docker start sonarqube`; escáner `sonarsource/sonar-scanner-cli`; `python pruebas/informe_sonar.py`

Endpoints de la API v1

Todo endpoint salvo los públicos exige la cabecera X-API-Key con una clave del rol indicado o superior.

Método	Ruta	Rol	Para qué
GET	`/api/v1/salud`	pública	Estado, modelos en uso, último dato, ciclo en curso y revisión de la base
GET	`/api/v1/rutas`	lector	Rutas con datos y su rango de fechas
POST	`/api/v1/predictions/run`	operador	Predicción de 15 a 120 min con intervalo, explicación, recomendación y tramo crítico
GET	`/api/v1/predictions`	lector	Predicciones guardadas (ruta, horizonte, fechas, modelo, ejecución)
GET	`/api/v1/predictions/{id}/segments`	lector	Predicción desagregada por segmento
POST	`/api/v1/recommendations/calculate`	operador	Recomendaciones de frecuencia por hora
GET	`/api/v1/recommendations`	lector	Recomendaciones guardadas
GET	`/api/v1/segments[/profile]`	lector	Segmentos de la ruta y perfil por tipo de día y hora
GET	`/api/v1/alerts`	lector	Alertas (estado, severidad, código, ruta, fechas)
GET	`/api/v1/alerts/summary`	lector	Volumen por código y estado y porcentaje de descartadas
GET	`/api/v1/alerts/{id}`	lector	Alerta con historial y notificaciones
PATCH	`/api/v1/alerts/{id}/status`	operador	Reconocer, asignar, cerrar o descartar, con comentario
POST	`/api/v1/frequency/executions`	operador	Lanza el ciclo incremental en segundo plano (202)
GET	`/api/v1/frequency/executions[/{id}]`	lector	Ejecuciones y detalle con resumen de calidad
GET	`/api/v1/data-quality`	lector	Problemas de calidad por ejecución, regla o dimensión
GET	`/api/v1/notifications`	lector	Notificaciones registradas
GET	`/api/v1/models`	lector	Registro de modelos y cuáles están en memoria
POST	`/api/v1/models/{id}/activate`	administrador	Activa un modelo verificando su integridad
GET / PUT	`/api/v1/parameters[/{clave}]`	lector / admin.	Parámetros vigentes y nueva versión
GET	`/api/v1/export/{recurso}`	lector	CSV o Excel de predicciones, recomendaciones, alertas, ejecuciones o calidad
GET	`/metrics` · `/tablero` · `/docs`	pública	Métricas, tablero y OpenAPI (docs solo fuera de producción)

Verificación del cierre (29 de octubre de 2026)

Calibración de alertas

Reglas anteriores y nuevas evaluadas sobre las mismas 528 recomendaciones de la demostración de diciembre. ALT-001 pasa a la condición literal del §5.2 (supera la capacidad disponible) y ALT-003 exige un déficit de al menos 2 buses o 15 %. La reducción es de 14 %: el volumen restante refleja que, con la capacidad registrada de 50 pasajeros y la ocupación objetivo de 80 %, la oferta actual queda corta en muchas horas pico; la carga

medida por segmento muestra que esa capacidad se supera en la realidad.

Alerta	Antes	Después
ALT-001	222	151
ALT-002	136	136
ALT-003	274	252
ALT-004	37	37
Total	669	576

Ciclo con la base del reto y despliegue en Docker

Ciclo por consola con origen base (12 de diciembre de 2025): 235 franjas y 527 viajes leídos de la base del reto en solo lectura, 62 problemas de calidad, segmentos de 502 viajes, 384 predicciones, 48 recomendaciones y 94 s en total. Subidas y bajadas idénticas a la copia local en las 235 franjas. Docker Compose: la API se construye, aplica las migraciones (revisión 0006) y queda sana. Sin clave responde 401; el ciclo lanzado por la API respondió 202 y un segundo lanzamiento mientras corría, 409. El ciclo asíncrono terminó exitosa en 71 s: leyó 225 franjas de la base del reto desde el contenedor, actualizó segmentos de 319 viajes y generó 384 predicciones; el monitoreo, con 4 días evaluables, dio un WAPE de 11.4 % a 30 min, 13.3 % a 60 min, 13.0 % a 120 min. El servicio programador arranca con su expresión cron y se detuvo tras verificarlo.

Tablero en el navegador

Prueba con Microsoft Edge (Playwright) contra la API en Docker: pide la clave, predice (8 franjas, 2 recomendaciones con tramo crítico), navega las pestañas con el teclado, abre el detalle de una alerta con su historial, lista ejecuciones, modelos (3 en uso) y parámetros y descarga la exportación. Sin errores de consola, sin desborde horizontal a 375 px y con contraste mínimo de 5,69:1 en los temas claro y oscuro (WCAG AA exige 4,5:1). La prueba encontró dos defectos que se corrigieron: el 404 del ícono y el intervalo extrapolado en franjas sin servicio.

Rendimiento

Prueba de carga con 10 usuarios concurrentes durante 62 s contra la API en Docker: 1076 peticiones, 17.3 por segundo, 0 errores. El p95 máximo de las consultas paginadas es 146 ms (RNF-002 exige menos de 2 s). La predicción bajo demanda calcula variables sobre un año de historia y, con 10 usuarios a la vez en un solo proceso, tarda unos 4,5 s; sola, alrededor de 1,1 s.

Consulta	Peticiones	p50 ms	p95 ms	p99 ms	Errores
alertas filtradas	127	89	129	149	0
ejecuciones	138	89	129	166	0
perfil por segmento	122	81	119	138	0
prediccion bajo demanda	121	4.394	4.736	4.971	0
predicciones paginadas	139	94	146	160	0
predicciones por ruta	163	97	144	160	0
recomendaciones paginadas	127	94	138	185	0
resumen de alertas	139	75	113	123	0

Volumen de ingesta (RNF-003) en una base temporal creada y eliminada por la prueba: 808.345 registros en 149 s (5.438 por segundo), pico de memoria de 964 MB; migraciones sobre base vacía en 0,9 s. Una segunda carga idéntica deja el mismo número de filas: no duplica. EXPLAIN (ANALYZE, BUFFERS) de las consultas incrementales de un día en la base del reto (solo lectura: on, timeout 5min): viajes del rango en 5,7 ms y lecturas por segmento en 130 ms, todas con los índices existentes y sin lecturas secuenciales sobre los 176 millones de lecturas, por lo que no hace falta pedir índices. El conteo agregado tardaba 5,6 s porque recorría 541 000 filas de progvehiculos; con el filtro adicional baja a 0,9 s con resultado idéntico.

SonarQube

Análisis con SonarQube Community Build 26.10 en Docker local y el escáner oficial. El primer análisis encontró 17 hallazgos: 1 bloqueante (recursión sin caso base explícito en pruebas/explain_fuente.py), 2 críticos (SMTP sin validación de certificado cuando se desactivaba TLS; sort sin comparador en el tablero) y 14 mayores (tablas

sin encabezado, tabindex en un elemento no interactivo, promesas sin manejar, expresión regular ambigua y generador pseudoaleatorio en la prueba de carga). Se corrigieron todos.

Medida final	Valor
Puerta de calidad	OK
Hallazgos abiertos	0
Bugs · vulnerabilidades · deuda técnica	0 · 0 · 0
Puntos de seguridad	0
Calificación de confiabilidad, seguridad y mantenibilidad	A · A · A
Duplicación	0.0 %
Cobertura (sin migraciones, pruebas/ ni JavaScript)	74.2 %
Líneas de código	5851

Segmentos

Validación contra timbradas_segmentos del operador: 17 de enero de 2023, 57 303 pares viaje-segmento con 99,73 % de subidas iguales y 99,82 % de bajadas, totales idénticos (45 744); 11 de febrero, 50 642 pares con 99,67 % y 99,77 %, totales idénticos (39 129). Perfil con las cuatro semanas de noviembre de 2025 (luego extendido a diciembre por el ciclo): 18 647 filas para 381 de los 386 segmentos. Factor de carga = mediana de carga máxima sobre subidas por viaje; carga máxima por viaje en percentiles 50, 90 y 99.

Ruta	Viajes	Factor de carga	Carga máx. p50 · p90 · p99	Viajes sobre 50
2	3.505	0,389	27 · 49 · 62	9 %
16	6.502	0,432	35 · 56 · 64	20 %
21	2.173	0,375	29 · 47 · 58	7 %

Retención y artículo

La retención archivó los 3 paquetes rechazados y movió los 6 archivos XGBoost de la fase 1 a modelos/archivo/; no se borró nada. El artículo (paper/main.tex) se actualizó a LightGBM con la comparación de familias, el ajuste con Optuna, el intervalo calibrado, el diagnóstico de residuos, la carga por tramo y el despliegue con ciclo incremental, y compila sin advertencias (13 páginas).

Comparación de familias de gradient boosting

Experimento previo a la elección del modelo, a 60 minutos, con las mismas 22 variables, la misma partición temporal y parada temprana en validación. Se siguió la guía de LightGBM y Gradient Boosting del curso (lihkir.github.io/SpatioTemporalAI/lightgbm.html).

Modelo (h = 60 min)	WAPE	Desv. Poisson	MAE	Árboles	Segundos
LightGBM Poisson (elegido)	10,69 %	3,954	17,53	1613	11
LightGBM con categóricas	10,69 %	3,978	17,54	1861	13
LightGBM Tweedie 1,3	10,72 %	4,001	17,58	760	7
XGBoost Poisson (fase 1)	10,73 %	3,977	17,60	2306	24
HistGradientBoosting (sklearn)	10,81 %	4,013	17,73	1000	41
GradientBoosting clásico (sklearn)	11,04 %	4,595	18,11	400	222

Ablación de variables candidatas (movilidad, hora pico y demanda reciente de las otras rutas): las mejoras son de centésimas de punto de WAPE, dentro de la variación entre ejecuciones. Se mantienen las 22 variables v3; la demanda de otras rutas además obligaría a predecir las tres rutas juntas.

Variables	WAPE	Desv. Poisson	Árboles
v3	10,69 %	3,954	1613
v3+moviles	10,68 %	3,960	2153
v3+moviles+pico	10,66 %	3,944	1796
v3+moviles+pico+otras_rutas	10,66 %	3,939	2476

Resultados del entrenamiento

Entrenamiento del 2026-10-29 (versión 20261029T033100) con datos hasta el 1 de diciembre de 2025 y 15 trials de Optuna por horizonte. Prueba: el 20 % final de los instantes (44.626 franjas), nunca usado para ajustar ni calibrar. El modelo que se guarda se reentrena después con todas las franjas.

Error de predicción en prueba (WAPE y desviación de Poisson)

Horizonte	Prom. 4 sem.	Sem. pasada	XGBoost	LightGBM	Desv. XGB	Desv. LGBM	RMSLE
30 min	14,9 %	18,4 %	10,2 %	**10,1 %**	3,66	3,57	0,272
60 min	14,9 %	18,4 %	10,5 %	**10,5 %**	3,83	3,85	0,290
120 min	14,9 %	18,4 %	10,9 %	**10,8 %**	4,22	4,23	0,301

Intervalo de confianza del 80 %

CQR lleva la cobertura al nivel nominal con un ancho apenas mayor que los cuantiles sin calibrar y con mejor interval score que el ancho constante, porque el intervalo se ensancha en las horas de más demanda en lugar de usar el mismo ancho todo el día.

Horizonte	Método	Cobertura	Debajo	Encima	Ancho	Interval score
30 min	Cuantiles sin calibrar	77,5 %	11,4 %	11,1 %	52,5	77,9
30 min	Cuantiles + CQR (en uso)	81,6 %	8,9 %	9,5 %	55,8	77,8
30 min	Ancho constante	81,1 %	9,5 %	9,4 %	53,3	84,7
60 min	Cuantiles sin calibrar	77,5 %	11,3 %	11,1 %	54,0	81,3
60 min	Cuantiles + CQR (en uso)	82,0 %	8,6 %	9,4 %	57,8	81,3
60 min	Ancho constante	80,9 %	9,6 %	9,4 %	55,0	89,3
120 min	Cuantiles sin calibrar	78,2 %	11,3 %	10,5 %	55,9	84,6
120 min	Cuantiles + CQR (en uso)	82,5 %	8,6 %	8,8 %	59,9	84,7
120 min	Ancho constante	81,0 %	9,3 %	9,7 %	56,3	93,2

Ajuste, sobreajuste e importancia

Horiz.	Árboles	Tasa	Hojas	Mín. hoja	Desv. CV base → ajust.	d Cohen	Mayor |SHAP| medio
30 min	2561	0,018	152	62	4,030 → 3,985	0,12	prom_4_semanas, lag_h, minuto_dia
60 min	1932	0,026	154	95	4,586 → 4,500	0,16	prom_4_semanas, minuto_dia, lag_h
120 min	1868	0,026	154	95	5,128 → 4,971	0,16	prom_4_semanas, minuto_dia, viajes_habituales

Error por ruta, tipo de día y hora

WAPE en % del modelo y, entre paréntesis, de la línea base (promedio de las 4 semanas previas).

Horizonte	Ruta 2	Ruta 16	Ruta 21	Día normal	Día especial	Pico	Valle
30 min	10,6 (15,4)	8,4 (13,4)	14,4 (18,6)	10,0 (13,3)	14,3 (99,2)	9,3 (14,4)	10,7 (15,2)
60 min	11,0 (15,4)	8,9 (13,4)	14,7 (18,6)	10,4 (13,3)	15,0 (99,2)	9,8 (14,4)	11,1 (15,2)
120 min	11,4 (15,4)	9,2 (13,4)	15,2 (18,6)	10,8 (13,3)	15,7 (99,2)	10,3 (14,4)	11,3 (15,2)

Residuos

Ljung-Box rechaza la independencia de los residuos en todas las rutas y horizontes, a 1 hora y a 1 día: queda autocorrelación de corto plazo (acf(1) entre 0,17 y 0,38, mayor a más horizonte). McLeod-Li rechaza la varianza constante a 1 hora en todos los casos y a 1 día en la mayoría. Las predicciones y la cobertura marginal del intervalo siguen siendo válidas, pero los errores de franjas vecinas no son independientes: un desvío en una franja tiende a repetirse en la siguiente.

Horizonte	Ruta	acf(1)	LB p (4)	LB p (96)	McLeod-Li p (4)	McLeod-Li p (96)
30 min	2	0,17	1.2e-109	5.6e-93	2.6e-07	1.0e+00
30 min	16	0,22	4.5e-162	5.1e-148	2.5e-42	1.3e-09
30 min	21	0,19	1.5e-113	1.5e-98	3.6e-11	9.9e-01
60 min	2	0,25	0.0e+00	1.1e-305	3.5e-08	1.0e+00
60 min	16	0,30	0.0e+00	0.0e+00	1.9e-109	1.3e-59
60 min	21	0,24	1.4e-296	5.7e-284	5.5e-20	3.5e-02
120 min	2	0,32	0.0e+00	0.0e+00	8.4e-36	1.9e-08
120 min	16	0,38	0.0e+00	0.0e+00	3.5e-172	1.1e-113
120 min	21	0,29	0.0e+00	0.0e+00	6.5e-47	3.5e-17

Demostración del ciclo incremental

Simulación del §14.2 con origen archivos: tres ciclos sobre la base ai_frecuencias, que tenía datos hasta el 1 de diciembre de 2025, tomando de datos/ los días siguientes como si llegaran de la base del reto. En cada día nuevo y en cada ruta se predice a las 5:00, 6:00, ... 20:00 y se guarda la recomendación de la hora siguiente. Los tres ciclos agregaron 2 328 franjas y 4 795 viajes. No hubo franjas inválidas; los 447 problemas de calidad fueron viajes de más de 6 horas (CAL-103), que se marcan y no generan alerta.

Ciclo	Días	Reentrenar	Franjas	Predicc.	Recom.	Alertas	Duración
15	2 al 4 dic	nunca	705	1 152	144 (123 viables)	228 nuevas	171 s
17	5 al 8 dic	siempre	923	1 536	192 (175 viables)	207 nuevas	473 s
19	9 al 11 dic	siempre	700	1 536	192 (171 viables)	198 nuevas, 48 actualizadas	458 s

Monitoreo del modelo

WAPE móvil de 7 días sobre las predicciones guardadas al cierre del ciclo 19: 10,1 % a 30 min, 10,3 % a 60 min y 11,0 % a 120 min, en línea con la prueba del entrenamiento y por debajo de la tolerancia de 15 %; no se generó ALT-010.

Campeón/retador

En el ciclo 17 la comparación era inválida: la ventana empezaba en el corte del modelo activo, el retador salió idéntico (WAPE 10,14 % en ambos a 30 min) y el empate activó tres modelos. Se corrigió la regla (ver reentrenamiento.py), esos modelos se rechazaron y se reactivaron los del entrenamiento completo. En el ciclo 19 el retador se entrenó con 934 franjas más que el activo (1 al 4 de diciembre) y ambos se evaluaron del 5 al 11 de diciembre, días que ninguno vio. El retador mejoró en los tres horizontes: WAPE 10,16 % frente a 10,26 % a 30 min, 10,64 % frente a 10,69 % a 60 min y 10,82 % frente a 10,93 % a 120 min, con menor desviación de Poisson y muy por debajo de la línea base (22,0 %). Se reentrenó con todos los datos hasta el 11 de diciembre y quedó activo (model_registry 38 a 40, que son los modelos en uso al cerrar este documento).

Idempotencia

El ciclo 19 volvió a procesar las horas del 8 de diciembre que ya había procesado el 17: las predicciones se actualizaron en lugar de duplicarse y 48 alertas abiertas se actualizaron con la misma clave de deduplicación.

Recomendaciones y volumen de alertas

En los 10 días, la frecuencia recomendada promedio fue de 8,8 buses por hora (headway de 8,5 min) frente a 8,5 buses observados. Por ruta: 7,1 recomendados frente a 7,8 observados en la ruta 2, 13,8 frente a 13,0 en la 16 y 5,6 frente a 4,7 en la 21. Con la ocupación objetivo de 80 % y la capacidad medida de 50 pasajeros se generaron 633 alertas en 528 horas-ruta: ALT-003 (oferta bajo la recomendada) 255, ALT-001 (ocupación alta prevista) 209, ALT-002 (ocupación crítica) 129, ALT-004 (baja utilización) 29, ALT-005 y ALT-007 (demanda fuera del intervalo) 11. Son 147 críticas, 457 preventivas y 29 informativas. Ninguna se notificó porque no hay destinatarios configurados.

Catálogos

Reglas de calidad de datos

Regla	Dimensión	Severidad	Tratamiento	Descripción
CAL-001	completitud	error	cuarentena	Campos obligatorios nulos
CAL-002	validez	error	excluido	Subidas o bajadas negativas
CAL-003	validez	error	excluido	Lectura imposible: mas de 90 subidas en una lectura del sensor
CAL-004	completitud	error	excluido	Dia sin lecturas del sensor: menos del 5 % de las lecturas habituales
CAL-005	unicidad	advertencia	reportado	Franja repetida en el lote
CAL-006	relacional	error	cuarentena	Ruta que no existe en la tabla rutas
CAL-007	consistencia	advertencia	marcado	Lecturas con cantidad negativa (reinicio del contador)
CAL-008	consistencia	advertencia	marcado	Bajadas del dia mas de 1.5 veces las subidas
CAL-101	validez	advertencia	marcado	Fin del viaje anterior a su inicio
CAL-102	consistencia	advertencia	marcado	Viaje cancelado registrado como finalizado
CAL-103	validez	advertencia	marcado	Duracion del viaje mayor a 360 minutos
CAL-104	relacional	error	cuarentena	Viaje de una ruta que no existe en la tabla rutas
CAL-201	validez	advertencia	excluido	Lecturas del sensor a mas de 300 m de la ruta: no se asignan a un segmento
CAL-202	consistencia	advertencia	marcado	Viaje sin bajadas registradas: no se puede estimar la carga a bordo por segmento

Alertas

Código	Severidad	Se genera cuando
ALT-001	preventiva o crítica	Ocupación prevista (escenario alto) sobre el máximo con la oferta actual
ALT-002	crítica	Ocupación esperada con la oferta actual sobre el umbral crítico (90 %)
ALT-003	preventiva	Buses por hora operados por debajo de los recomendados
ALT-004	informativa	Ocupación bajo el mínimo (30 %) con más buses de los necesarios
ALT-005	preventiva	Demanda observada bajo el intervalo y con caída mayor a la tolerancia
ALT-006	crítica	Hueco entre despachos mayor a dos headways
ALT-007	preventiva	Demanda observada sobre el intervalo y con aumento mayor a la tolerancia
ALT-008	preventiva	Registros atípicos en el consolidado (CAL-003, CAL-007, CAL-008)
ALT-009	crítica	Día sin lecturas del sensor (CAL-004) o datos atrasados
ALT-010	crítica	WAPE móvil de 7 días del modelo sobre la tolerancia

Pendiente u observado

- Capacidad y umbrales: la carga medida supera la capacidad registrada de 50 pasajeros en 7 a 20 % de los viajes. SOBUSA debe confirmar la capacidad real, la ocupación objetivo y las tolerancias de alerta (marcadas como no validadas); con esos valores el volumen de alertas cambiará. Un headway máximo de 5 min no es viable con la flota observada.
- Envío real de notificaciones: el código está listo; faltan las credenciales de SMTP o de WhatsApp Business (y una plantilla aprobada para escribir fuera de la ventana de 24 h), que no deben ir en el repositorio.
- Sentido de la ruta: los datos no distinguen ida y vuelta; los segmentos recorren la ruta completa y el tramo crítico de la mañana y de la tarde cae en mitades distintas del recorrido.
- Predicción bajo demanda con varios usuarios a la vez: unos 4,5 s con 10 concurrentes en un solo proceso; se puede escalar con más procesos de uvicorn si el tablero lo requiere.
- Historial de git: .env sigue en el historial y en el repositorio remoto (host y usuario de la base, no la contraseña). Limpiarlo exige reescribir el historial y forzar la subida, coordinado con el equipo.
- SonarQube en CI necesita un servidor accesible desde GitHub y los secretos SONAR_TOKEN y SONAR_HOST_URL; el análisis local está documentado arriba.

Paquete frecuencias: base

frecuencias/config.py

24 líneas · modificado · 1 nota

Descripción del archivo · modificado · 28 oct 2026

Se agregó la zona horaria de la operación. El resto de constantes no cambió.

Línea 24 · nota nueva · 28 oct 2026

```
ZONA_HORARIA = "America/Bogota"
```

Hora local de Barranquilla. Los datos del reto vienen en hora local sin zona; la base ai_frecuencias guarda timestamptz, así que toda conversión pasa por esta constante.

frecuencias/tiempo.py

19 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 28 oct 2026

Conversión entre la hora local sin zona (la de los datos y los modelos) y la hora con zona que guarda PostgreSQL. Centraliza la regla para que ningún módulo convierta por su cuenta.

Línea 8 · nota nueva · 28 oct 2026

```
def ahora_utc() -> datetime:
```

Marca de tiempo del sistema en UTC; se usa para auditoría (creada_en, actualizada_en).

Línea 12 · nota nueva · 28 oct 2026

```
def a_zona(ts) -> pd.Timestamp:
```

Hora local sin zona a hora con zona de Bogotá; si ya trae zona solo la convierte.

Línea 17 · nota nueva · 28 oct 2026

```
def a_local(ts) -> pd.Timestamp:
```

Inversa de a_zona: lo que llega de la base vuelve a hora local sin zona para cruzarlo con la serie y las variables del modelo.

frecuencias/observabilidad.py

93 líneas · nuevo · 7 notas

Descripción del archivo · nuevo · 29 oct 2026

Observabilidad de la solución (§11.4, RNF-006): logs estructurados en JSON, métricas Prometheus, el identificador de correlación que une la petición HTTP, los logs y la fila de execution_log, y el saneo de mensajes de error.

Línea 16 · nota nueva · 28 oct 2026

```
PETICIONES = Counter("frecuencias_peticiones_total", "Peticiones HTTP atendidas", ["metodo", "r...
```

Contadores e histogramas que expone /metrics: peticiones y duración por ruta, predicciones por horizonte, recomendaciones por viabilidad, alertas por código y severidad, y ejecuciones por tipo y estado.

Línea 22 · nota nueva · 29 oct 2026

```
CIRCUITOS = Gauge("frecuencias_circuito_abierto", "1 si el circuito de la integracion externa e...
```

Estado de cada circuit breaker (1 = abierto), latencia de los datos en horas y notificaciones por canal y estado.

Línea 28 · nota nueva · 28 oct 2026

```
def configurar_logs() -> None:
```

structlog emite una línea JSON por evento con nivel, hora UTC y el correlation_id de la petición en curso. El nivel se controla con LOG_LEVEL.

Línea 50 · nota nueva · 29 oct 2026

```
PATRONES_SENSIBLES = [
```

Un error de psycopg2 trae el host, la IP y el usuario de la base. Estos patrones los reemplazan por *** antes de guardar el error en execution_log, en una alerta o en un log: el host y el usuario de la base del reto nunca deben quedar registrados.

Línea 58 · nota nueva · 29 oct 2026

```
def describir_error(error: Exception, largo: int = 500) -> str:
```

Mensaje de error saneado y sin la sentencia SQL ni sus parámetros; lo usan el ciclo, la API, la ingesta y el programador.

Línea 70 · nota nueva · 28 oct 2026

```
def normalizar_correlation_id(valor: str | None) -> str:
```

Si el cliente manda un X-Correlation-Id que no es un UUID se genera uno nuevo: la columna execution_log.correlation_id es de tipo UUID y un valor arbitrario rompería el insert.

Línea 77 · nota nueva · 28 oct 2026

```
class MiddlewareCorrelacion(BaseHTTPMiddleware):
```

Middleware que asigna el correlation_id, lo devuelve en la cabecera de la respuesta y registra método, ruta, estado y duración de cada petición.

frecuencias/seguridad.py

87 líneas · nuevo · 5 notas

Descripción del archivo · nuevo · 29 oct 2026

Autenticación de la API con claves por rol (§7.2, §18). La clave se muestra una sola vez al crearla; en la base solo se guarda su huella SHA-256 y un prefijo para identificarla.

Línea 12 · nota nueva · 29 oct 2026

```
ROLES = {"lector": 1, "operador": 2, "administrador": 3}
```

Tres roles jerárquicos: lector (consultas y exportación), operador (predecir, recomendar, gestionar alertas y lanzar el ciclo) y administrador (activar modelos y publicar parámetros).

Línea 22 · nota nueva · 29 oct 2026

```
def generar() -> str:
```

Clave aleatoria de 32 bytes con el módulo secrets y el prefijo fq_.

Línea 39 · nota nueva · 29 oct 2026

```
def validar(conn: Connection, clave: str | None) -> dict | None:
```

Busca la huella entre las claves activas. El último uso se actualiza como mucho cada 5 minutos para no escribir en cada petición.

Línea 52 · nota nueva · 29 oct 2026

```
def revocar(conn: Connection, nombre: str) -> bool:
```

Revocar deja la clave inactiva con fecha; no se borra, queda la trazabilidad.

Línea 63 · nota nueva · 29 oct 2026

```
def main():
```

CLI: python -m frecuencias.seguridad crear –nombre ... –rol ..., listar y revocar. En Docker: docker compose exec api python -m frecuencias.seguridad crear ...

frecuencias/resiliencia.py

72 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 29 oct 2026

Resiliencia para integraciones externas (RNF-010): reintento controlado con espera exponencial y circuit breaker. Lo usan la lectura de la base del reto y las notificaciones.

Línea 12 · nota nueva · 29 oct 2026

```
class Interruptor:
```

Circuit breaker: tras N fallos seguidos se abre y rechaza las llamadas sin intentarlas; pasado el tiempo de espera deja pasar una (semiabierto) y se cierra si funciona.

Línea 60 · nota nueva · 29 oct 2026

```
def reintentar(funcion: Callable, intentos: int = 3, espera_inicial_s: float = 1.0, factor: flo...
```

Reintenta solo las excepciones transitorias indicadas, con espera 1, 2, 4... segundos, y devuelve cuántos intentos hizo. Un circuito abierto no se reintenta.

Ingesta, histórico y calidad de datos

frecuencias/ingesta/consultas.py

65 líneas · modificado · 5 notas

Descripción del archivo · modificado · 29 oct 2026

Consultas de solo lectura a la base del reto.

Línea 20 · nota nueva · 29 oct 2026

```
WHERE c.idprogramacion = ANY(:ids) AND p.idprogramacion = ANY(:ids)
```

Optimización por EXPLAIN: el filtro repetido sobre progvehiculos evita recorrer 541 000 filas por su llave primaria solo para obtener la ruta. La consulta de un día pasó de 5,4 s a 0,9 s con resultado idéntico.

Línea 39 · nota nueva · 28 oct 2026

```
QUERY_VIAJES_RANGO = QUERY_VIAJES + """
```

Viajes con fechasalida dentro de [desde, hasta). Sigue filtrando por idprogramacion para no recorrer los ~176 millones de lecturas de conteo_pasajeros_destino.

Línea 43 · nota nueva · 29 oct 2026

```
QUERY_SEGMENTOS = """
```

Segmentos activos de las tres rutas (rutas_segmentos de la configuración activa) con su geometría en WKT.

Línea 52 · nota nueva · 29 oct 2026

```
QUERY_LECTURAS_SEGMENTO = """
```

Lecturas con subidas o bajadas de los viajes del rango, con coordenadas y contadores acumulados.

Línea 60 · nota nueva · 29 oct 2026

```
QUERY_TIMBRADAS_SEGMENTOS = """
```

Salida del operador por viaje y segmento, para validar la reproducción de sus scripts.

frecuencias/ingesta/historico.py

211 líneas · nuevo · 7 notas

Descripción del archivo · nuevo · 29 oct 2026

Histórico analítico propio (§8.2, RF-013): copia validada de la demanda por franja (demand_history) y de los

viajes (trip_history) dentro de ai_frecuencias. La API y el ciclo leen de aquí y la base del reto solo se consulta para traer lo nuevo.

Línea 18 · nota nueva · 28 oct 2026

```
MARGEN_VIAJES = pd.Timedelta(hours=48)
```

La extracción incremental pide viajes desde 48 h antes del inicio del rango: una subida se cuenta en la franja en que ocurrió, pero el viaje pudo salir el día anterior. Con 8 h de margen la copia difería en 2 subidas de 10 franjas; con 48 h es idéntica a la local.

Línea 20 · nota nueva · 29 oct 2026

```
FUENTE = Interruptor("base_reto", fallos_maximos=3, espera_s=300.0)
```

Circuit breaker de la base del reto: tres fallos seguidos lo abren durante 5 min.

Línea 37 · nota nueva · 28 oct 2026

```
def upsert(conn: Connection, tabla, df: pd.DataFrame, llave: list[str]) -> int:
```

Carga masiva con COPY a una tabla temporal y luego INSERT ... ON CONFLICT DO UPDATE. Repetir una carga no duplica filas (§14.2). Medido en la prueba de volumen: 808 345 registros en 149 s; un día incremental tarda ~0,3 s. Ahora es pública porque la usa también la capa de segmentos.

Línea 87 · nota nueva · 28 oct 2026

```
def cargar(conn: Connection, demanda_df: pd.DataFrame, viajes_df: pd.DataFrame | None, executio...
```

Valida la demanda y los viajes con las reglas de calidad, guarda las filas (las inválidas quedan marcadas con valido = false y su motivo) y registra cada problema en data_quality_issue. Devuelve el resumen y la lista de problemas para generar alertas.

Línea 105 · nota nueva · 28 oct 2026

```
def leer_demanda(conn: Connection, desde=None, hasta=None, solo_validas: bool = True) -> pd.Dat...
```

Devuelve solo las franjas válidas, con las mismas columnas y tipos que limpiar_demanda: la serie que sale de la base es idéntica a la que se usaba desde el parquet.

Línea 157 · nota nueva · 28 oct 2026

```
def extraer_rango(motor: Engine, desde: pd.Timestamp, hasta: pd.Timestamp) -> tuple[pd.DataFram...
```

Extracción incremental desde la base del reto, con reintento ante errores de conexión y el circuit breaker. Validada: el 12 de diciembre de 2025 leído de la base coincide en subidas y bajadas con la copia local en las 235 franjas.

Línea 174 · nota nueva · 28 oct 2026

```
def carga_inicial(motor_destino: Engine, hasta: pd.Timestamp | None = None) -> dict:
```

Carga inicial desde datos/ (la copia local del histórico 2023-2025) con su fila en execution_log y la marca de agua.

frecuencias/analitica/calidad.py

134 líneas · reescrito · 6 notas

Descripción del archivo · reescrito · 29 oct 2026

Reglas de calidad (§6, RF-004) como catálogo con dimensión, severidad y tratamiento. limpiar_demanda conserva exactamente su resultado anterior (243 001 franjas), y validar_demanda y validar_viajes devuelven además cada problema encontrado para guardarlo y auditarlo.

Línea 11 · nota nueva · 28 oct 2026

```
REGLAS = {
```

Catálogo CAL-001 a CAL-008 (demanda), CAL-101 a CAL-104 (viajes) y CAL-201 y CAL-202 (segmentos). El tratamiento indica qué pasa con el registro: cuarentena, excluido, marcado o reportado.

Línea 28 · nota nueva · 29 oct 2026

```
"CAL-201": ("validez", "advertencia", "excluido",
```

Lecturas a más de 300 m de la ruta (GPS desviado): no se asignan a ningún segmento. En noviembre de 2025 fueron 1,9 % de las lecturas.

Línea 37 · nota nueva · 28 oct 2026

```
def _mascaras(df: pd.DataFrame) -> tuple[pd.Series, pd.Series]:
```

Las dos reglas que ya usaba el modelo (día sin lecturas del sensor y lectura imposible) en una sola función, compartida por limpiar_demanda y validar_demanda para que no diverjan.

Línea 55 · nota nueva · 29 oct 2026

```
def problema(regla: str, fuente: str, clave: dict, valor=None, detalle: str | None = None) -> d...
```

Construye un problema de calidad con los datos del catálogo; es pública porque la usa también la capa de segmentos.

Línea 66 · nota nueva · 28 oct 2026

```
def validar_demanda(df: pd.DataFrame, rutas_validas: set[int] | None = None) -> tuple[pd.DataFr...
```

Aplica todas las reglas de demanda. Las franjas de un día sin lecturas o con lecturas imposibles se guardan como no válidas: el modelo las ve como faltantes, nunca como cero pasajeros.

Línea 113 · nota nueva · 28 oct 2026

```
def validar_viajes(df: pd.DataFrame, rutas_validas: set[int] | None = None) -> tuple[pd.DataFra...
```

Reglas de viajes. Con los datos completos: 93 viajes con fin anterior al inicio (CAL-101) y 18 082 con duración mayor a 6 h (CAL-103), que no se usan para estimar tiempos de ciclo.

Segmentos de ruta

frecuencias/segmentos/carga.py

296 líneas · nuevo · 9 notas

Descripción del archivo · nuevo · 29 oct 2026

Capa analítica por segmento (§6.1, §10.1, §11.2). Los scripts del operador no se pueden ejecutar sobre la base del reto (solo lectura, sin conteo_pasajeros ni rutasgeograficas, filtran el proceso cootransoriente y tienen errores de sintaxis), así que su lógica se reproduce en ai_frecuencias sin tocar la fuente: cada lectura va al segmento más cercano de su ruta y se agrega por viaje, segmento y bloque de 15 min. Validado contra timbradas_segmentos: 99,7 % de los pares viaje-segmento iguales y totales idénticos en dos días de 2023.

Línea 26 · nota nueva · 29 oct 2026

```
TOLERANCIA_SIMPLIFICACION = 0.000005
```

Los segmentos de 500 m están armados con porciones de 20 cm (~2 500 vértices, 40 KB). Simplificarlos a ~0,5 m deja ~10 vértices y la asignación pasa de 14 s a 0,5 s por 14 000 lecturas, con las mismas distancias.

Línea 32 · nota nueva · 29 oct 2026

```
ASIGNAR = """
```

Segmento más cercano con el índice GiST (operador <->, como el trigger del operador) y distancia en metros sobre geography. Calcula además la carga a bordo balanceada: las bajadas del viaje se escalan para que sumen lo mismo que las subidas.

Línea 58 · nota nueva · 29 oct 2026

```
AGREGAR = """
```

Agregación por viaje, segmento y franja. La columna pasajeros reproduce la fórmula del operador (diferencia de acumulados), que no sirve como carga: su mediana es 6 041. La carga útil es carga_media y carga_maxima.

Línea 80 · nota nueva · 29 oct 2026

```
def cargar_catalogo(conn: Connection, fuente: Engine) -> int:
```

Copia los 386 segmentos activos y analiza la tabla para que el planificador use el índice espacial.

Línea 115 · nota nueva · 29 oct 2026

```
def procesar_lote(conn: Connection, lecturas: pd.DataFrame,
```

Timeout propio de 10 min (el de 30 s es para la API) y sin JIT. Las lecturas lejanas generan CAL-201 y los viajes sin bajadas CAL-202.

Línea 160 · nota nueva · 29 oct 2026

```
def actualizar(destino: Engine, fuente: Engine, desde: pd.Timestamp, hasta: pd.Timestamp,
```

Proceso por lotes de 500 viajes con su ejecución en execution_log. Cuatro semanas: 12 927 viajes, 1,75 millones de lecturas y 790 178 observaciones en 146 s.

Línea 193 · nota nueva · 29 oct 2026

```
def validar_contra_operador(destino: Engine, fuente: Engine, fecha: pd.Timestamp) -> dict:
```

python -m frecuencias.segmentos.carga –validar AAAA-MM-DD compara con la salida del operador.

Línea 224 · nota nueva · 29 oct 2026

```
def construir_perfil(observaciones: pd.DataFrame, calendario) -> pd.DataFrame:
```

Perfil por ruta, segmento, tipo de día y hora: promedio, mínimo y máximo de subidas, bajadas y pasajeros, carga media, p90 y máxima, y cuota de subidas. Los viajes que pasaron sin subidas en un segmento cuentan como cero en el promedio y el mínimo.

Línea 258 · nota nueva · 29 oct 2026

```
def recalcular_perfil(conn: Connection, hasta: pd.Timestamp, execution_id: int | None,
```

El perfil se recalcula con los últimos 28 días después de cada actualización.

`frecuencias/segmentos/consultas.py`

85 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 29 oct 2026

Consultas de segmentos para la API.

Línea 17 · nota nueva · 29 oct 2026

```
def catalogo(conn: Connection, id_ruta: int | None = None, con_geometria: bool = False) -> list...
```

Segmentos de una ruta con su posición en km; con geometry=true agrega la geometría en GeoJSON.

Línea 41 · nota nueva · 29 oct 2026

```
def tramo_critico(conn: Connection, id_ruta: int, tipo_dia: str, hora: int, capacidad: float) -...
```

Segmento con mayor carga p90 a esa hora y tipo de día, y su ocupación respecto de la capacidad: explica dónde se llena el bus.

Línea 55 · nota nueva · 29 oct 2026

```
def desagregar(conn: Connection, prediction_id) -> dict:
```

Reparte la predicción de una franja entre los segmentos con la cuota histórica de subidas, usando el tipo de día exacto con que predijo el modelo (feature_snapshot).

Persistencia en ai_frecuencias

`frecuencias/persistencia/tablas.py`

333 líneas · nuevo · 6 notas

Descripción del archivo · nuevo · 29 oct 2026

Definición de las 16 tablas del esquema ai_frecuencias para SQLAlchemy Core. Refleja las migraciones 0001 a 0006; tests/test_esquema.py verifica que coinciden con la base.

Línea 22 · nota nueva · 28 oct 2026

```
UUID_ALEATORIO = text("gen_random_uuid()")
```

Las claves UUID las genera PostgreSQL (gen_random_uuid()).

Línea 25 · nota nueva · 29 oct 2026

```
class Geometria(UserDefinedType):
```

Tipo mínimo para la columna geometry de PostGIS, sin depender de GeoAlchemy2.

Línea 189 · nota nueva · 28 oct 2026

```
demanda = Table(
```

demand_history: una fila por ruta y franja, con el indicador de validez y el motivo.

Línea 255 · nota nueva · 29 oct 2026

```
segmento = Table(
```

route_segment, segment_observation y segment_profile (migración 0006).

Línea 310 · nota nueva · 29 oct 2026

```
clave_api = Table(
```

api_key (migración 0005).

Línea 324 · nota nueva · 28 oct 2026

```
instantanea = Table(
```

feature_snapshot: las variables exactas con que se hizo cada predicción (§9.3).

frecuencias/persistencia/consultas.py

20 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Paginación común de todos los listados de la API.

Línea 5 · nota nueva · 28 oct 2026

```
TAMANO_MAXIMO = 500
```

Ninguna página devuelve más de 500 filas; la exportación recorre páginas.

Línea 8 · nota nueva · 28 oct 2026

```
def paginar(conn: Connection, consulta: Select, tabla: Table, orden: str, descendente: bool,
```

Cuenta el total con la misma consulta filtrada y valida que la columna de orden exista.

frecuencias/persistencia/parametros.py

55 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Parámetros operativos versionados (§5.1, RF-008): cada cambio crea una versión nueva y deja la anterior como no vigente, con autor y fecha.

Línea 11 · nota nueva · 28 oct 2026

```
def vigentes(conn: Connection, id_ruta: int | None = None) -> dict[str, float | None]:
```

Valores vigentes como diccionario clave → valor. El valor de la ruta reemplaza al general.

Línea 32 · nota nueva · 28 oct 2026

`def publicar(conn: Connection, clave: str, valor: float | None, usuario: str, id_ruta: int | No...`

Nueva versión de un parámetro. Si la clave no existe exige unidad y descripción.

frecuencias/persistencia/ejecuciones.py

51 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 29 oct 2026

Bitácora de ejecuciones (execution_log) con estado, conteos, marca de agua y correlation_id (RF-013).

Línea 11 · nota nueva · 28 oct 2026

`def iniciar(conn: Connection, tipo: str, detalle: dict | None = None, correlation_id=None) -> d...`

Crea la fila en estado en_curso y devuelve su id y correlation_id.

Línea 24 · finalizar() · nota nueva · 29 oct 2026

`valores = {"estado": estado, "finalizada_en": func.clock_timestamp(), "registros_leidos": regis...`

Corrección: la hora de fin la pone el reloj de PostgreSQL, igual que la de inicio. Con el reloj de Windows, una ejecución que empezaba y terminaba en el mismo milisegundo podía violar la restricción fin >= inicio por el desfase con la máquina virtual de Docker (fallo intermitente en las pruebas).

Línea 28 · finalizar() · nota nueva · 29 oct 2026

`valores["detalle"] = func.coalesce(ejecucion.c.detalle, literal_column("'{}'::jsonb")).op("||")(`

Corrección: el detalle final se combina con el inicial (operador || de JSONB) en lugar de reemplazarlo; antes se perdían el origen, el ciclo padre y quién la solicitó.

Línea 38 · nota nueva · 28 oct 2026

`def ultima_marca_agua(conn: Connection, tipo: str = "ingesta") -> datetime | None:`

Hasta dónde llegó la última ingesta exitosa.

frecuencias/persistencia/registro_modelos.py

65 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Registro de modelos (§9.2, RF-018 y RF-019): un solo modelo activo por horizonte.

Línea 40 · nota nueva · 28 oct 2026

`def activar(conn: Connection, id_modelo: int, usuario: str) -> dict:`

Retira el activo del mismo horizonte y activa el candidato, con fecha y usuario que aprueba.

Línea 58 · nota nueva · 28 oct 2026

`def rechazar(conn: Connection, id_modelo: int, usuario: str) -> dict:`

Marca un candidato como rechazado (no se borra: queda la trazabilidad).

frecuencias/persistencia/predicciones.py

52 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Guardado y consulta de predicciones.

Línea 10 · nota nueva · 28 oct 2026

`CLAVE = [`

La misma expresión del índice único de la migración 0001: ON CONFLICT la necesita idéntica.

Línea 24 · nota nueva · 28 oct 2026

```
def guardar(conn: Connection, filas: list[dict]) -> list:
```

Upsert: volver a predecir la misma franja con el mismo modelo actualiza la fila.

frecuencias/persistencia/recomendaciones.py

31 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 28 oct 2026

Guardado y consulta de recomendaciones de frecuencia, con filtros por ruta, fechas, viabilidad y ejecución.

frecuencias/persistencia/alertas.py

123 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 29 oct 2026

Alertas con deduplicación y ciclo de vida (§5.3, RF-009 a RF-011).

Línea 12 · nota nueva · 28 oct 2026

```
TRANSICIONES = {
```

Máquina de estados: abierta → reconocida/asignada → cerrada o descartada.

Línea 26 · nota nueva · 28 oct 2026

```
def registrar(conn: Connection, datos: dict) -> tuple[object, bool]:
```

Si ya hay una alerta abierta con la misma clave se actualiza; solo las nuevas registran historial y se notifican.

Línea 55 · nota nueva · 28 oct 2026

```
def cambiar_estado(conn: Connection, id_alerta, estado_nuevo: str | None, usuario: str,
```

Valida la transición, exige usuario asignado para pasar a asignada y guarda cada cambio en alert_history.

Línea 78 · nota nueva · 29 oct 2026

```
def resumen(conn: Connection, desde: datetime | None = None, hasta: datetime | None = None,
```

Volumen por código, severidad y estado, y porcentaje de alertas descartadas sobre las gestionadas: la medición de falsos positivos que piden el §9.4 y el §18.

frecuencias/persistencia/calidad.py

35 líneas · nuevo · 1 nota

Descripción del archivo · nuevo · 28 oct 2026

Consultas de problemas de calidad y de notificaciones para la API y el tablero.

Línea 20 · nota nueva · 28 oct 2026

```
def resumen(conn: Connection, execution_id: int) -> list[dict]:
```

Problemas de una ejecución agrupados por regla, dimensión y tratamiento.

Modelo: entrenamiento, intervalos y servicio

frecuencias/inferencia/evaluacion.py

74 líneas · modificado · 5 notas

Descripción del archivo · modificado · 28 oct 2026

Métricas nuevas para comparar familias de modelos y evaluar intervalos.

Línea 4 · nota nueva · 28 oct 2026

```
HORAS_PICO = {6, 7, 8, 16, 17, 18}
```

Horas pico para el desglose del error: 6 a 8 y 16 a 18 h.

Línea 17 · nota nueva · 28 oct 2026

```
def desviacion_poisson(y, yhat) -> float:
```

Desviación de Poisson: la pérdida del objetivo de conteo.

Línea 33 · nota nueva · 28 oct 2026

```
def pinball(y, cuantil, q: float) -> float:
```

Pérdida pinball de un cuantil.

Línea 38 · nota nueva · 28 oct 2026

```
def metricas_intervalo(y, inferior, superior, nivel: float) -> dict:
```

Cobertura, fracción por debajo y por encima, ancho medio e interval score.

Línea 63 · nota nueva · 28 oct 2026

```
def desglose(df: pd.DataFrame, y_col: str, pred_col: str, base_col: str | None = None) -> dict:
```

WAPE del modelo y de la línea base por ruta, tipo de día y hora pico o valle.

frecuencias/inferencia/ajuste.py

66 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 28 oct 2026

Ajuste de hiperparámetros de LightGBM con Optuna (lihkir.github.io/SpatioTemporalAI): validación temporal walk-forward y búsqueda bayesiana.

Línea 10 · nota nueva · 28 oct 2026

```
PARAMETROS_BASE = {
```

Objetivo de Poisson, determinista y con semilla fija.

Línea 19 · nota nueva · 28 oct 2026

```
def pliegues(datos: pd.DataFrame, n_splits: int = 4) -> list[tuple[np.ndarray, np.ndarray]]:
```

TimeSeriesSplit de 4 pliegues sobre instantes únicos, no sobre filas.

Línea 42 · nota nueva · 28 oct 2026

```
def ajustar(datos: pd.DataFrame, features: list[str], n_trials: int, n_splits: int = 4) -> dict:
```

TPE con semilla; el primer trial es la configuración base. Solo usa datos anteriores al corte de prueba.

Línea 48 · objetivo() · nota nueva · 28 oct 2026

```
"learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
```

Espacio de búsqueda: tasa, hojas, mínimo por hoja, submuestreo y regularización L2.

frecuencias/inferencia/cuantiles.py

53 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 28 oct 2026

Intervalo de confianza del 80 % (RF-006): tres modelos LightGBM cuantílicos calibrados con CQR.

Línea 7 · nota nueva · 28 oct 2026

`CUANTILES = (0.1, 0.5, 0.9)`

Cuantiles 0,1, 0,5 y 0,9; el intervalo nominal es del 80 %.

Línea 35 · nota nueva · 28 oct 2026

`def calibrar(y, P: np.ndarray, nivel: float = NIVEL) -> float:`

CQR: cuantil (n+1) · 0,8/n de cuánto se sale cada observación de validación.

Línea 43 · nota nueva · 28 oct 2026

`def intervalo(P: np.ndarray, ajuste: float, puntual=None) -> tuple[np.ndarray, np.ndarray]:`

Aplica el ajuste, recorta en cero y asegura que contenga la predicción puntual.

Línea 52 · nota nueva · 28 oct 2026

`def ancho_constante(y_val, puntual_val, nivel: float = NIVEL) -> float:`

Referencia de ancho fijo; CQR debe superarla en interval score.

frecuencias/inferencia/diagnostico.py

40 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 28 oct 2026

Diagnóstico del modelo que se guarda en el reporte de entrenamiento.

Línea 9 · nota nueva · 28 oct 2026

`def importancia(modelo: lgb.Booster, X: pd.DataFrame, muestra: int = 3000, semilla: int = 42) -...`

Importancia por ganancia y por |SHAP| medio (pred_contrib de LightGBM).

Línea 18 · nota nueva · 28 oct 2026

`def residuos(df: pd.DataFrame, y_col: str, pred_col: str) -> dict:`

Ljung-Box y McLeod-Li por ruta en rezagos de 1 h y 1 día.

Línea 34 · nota nueva · 28 oct 2026

`def sobreajuste(y_train, p_train, y_val, p_val) -> dict:`

Brecha entre error de entrenamiento y de validación, normalizada y como d de Cohen.

frecuencias/inferencia/artefactos.py

97 líneas · nuevo · 5 notas

Descripción del archivo · nuevo · 28 oct 2026

Paquete de modelo versionado (§9.2): modelo puntual, tres cuantiles, metadatos y manifiesto SHA-256.

Línea 20 · nota nueva · 28 oct 2026

`class ArtefactoAlterado(RuntimeError):`

Error si un archivo del paquete cambió después de registrado.

Línea 45 · predecir() · nota nueva · 28 oct 2026

`def predecir(self, X: pd.DataFrame) -> dict:`

Predicción puntual, límites del intervalo y contribuciones por variable.

Línea 58 · nota nueva · 28 oct 2026

`def guardar(carpeta: Path, punto: lgb.Booster, cuantiles: dict, meta: dict) -> tuple[Path, str]:`

Escribe los archivos y el manifiesto; la huella del manifiesto es la que se registra.

Línea 70 · nota nueva · 28 oct 2026

`def cargar(carpeta, sha_esperado: str | None = None, model_registry_id: int | None = None) -> P...`

Verifica la huella registrada y la de cada archivo antes de usar el modelo.

Línea 94 · nota nueva · 28 oct 2026

`def marcar_activo_local(horizonte: int, carpeta: Path, sha: str, model_registry_id: int | None)...`

modelos/activos.json: los paquetes activos cuando no hay base.

frecuencias/inferencia/prediccion.py

107 líneas · nuevo · 5 notas

Descripción del archivo · nuevo · 29 oct 2026

Predicción en servicio a partir de la serie conocida hasta la franja de referencia.

Línea 27 · nota nueva · 28 oct 2026

`def serie_conocida(ruta: pd.DataFrame, referencia: pd.Timestamp, hasta: pd.Timestamp) -> pd.Dat...`

Borra todo lo posterior a la referencia.

Línea 43 · nota nueva · 28 oct 2026

`def explicacion(contribuciones: np.ndarray, fila: pd.Series, features: list[str]) -> dict:`

Las cinco variables con mayor contribución y su efecto en %.

Línea 62 · modelo_para() · nota nueva · 28 oct 2026

`def modelo_para(self, minutos: int) -> int:`

Para cada antelación usa el menor horizonte entrenado que la cubre.

Línea 68 · perfil() · nota nueva · 28 oct 2026

`def perfil(self, ruta: pd.DataFrame, referencia: pd.Timestamp, hasta_min: int | None = None) ->...`

Ocho franjas de 15 min con valor puntual, intervalo, explicación, línea base y versiones.

Línea 86 · perfil() · nota nueva · 29 oct 2026

`if not habitual:`

Corrección encontrada en la prueba del tablero: en franjas sin servicio habitual (de madrugada) los modelos cuantílicos, entrenados solo con franjas con servicio, extrapolaban hasta 90 pasajeros y la recomendación pedía 5 buses a la 1 a. m. Ahí el intervalo pasa a ser la predicción puntual más o menos el ajuste conforme y se marca intervalo_calibrado = false.

frecuencias/inferencia/modelos.py

36 líneas · reescrito · 2 notas

Descripción del archivo · reescrito · 28 oct 2026

Carga de los modelos activos: primero del registro en la base y, si no hay base, de modelos/activos.json.

Línea 8 · nota nueva · 28 oct 2026

`def _validar(paquete: artefactos.Paquete) -> artefactos.Paquete:`

Rechaza un paquete entrenado con otra versión de variables.

Línea 26 · nota nueva · 28 oct 2026

`def cargar_modelos(motor=None) -> dict[int, artefactos.Paquete]:`

Si no hay modelos activos indica el comando para entrenar.

frecuencias/inferencia/entrenamiento.py

212 líneas · reescrito · 6 notas

Descripción del archivo · reescrito · 28 oct 2026

Entrenamiento con LightGBM: ajuste, evaluación frente a líneas base y XGBoost, intervalos, diagnóstico, paquete, registro y activación.

Línea 57 · nota nueva · 28 oct 2026

```
def evaluar_candidato(df: pd.DataFrame, features: list[str], parametros: dict, comparar: bool =...
```

Métricas, intervalos, cobertura por hora y ruta, desglose y diagnóstico.

Línea 77 · evaluar_candidato() · nota nueva · 28 oct 2026

```
ajuste_cqr = cq.calibrar(val["subidas"], P_val)
```

La calibración usa el periodo de validación, nunca el de prueba.

Línea 120 · nota nueva · 28 oct 2026

```
def construir_paquete(df: pd.DataFrame, horizonte: int, parametros: dict, evaluacion: dict, aju...
```

Modelo final reentrenado con todas las franjas y el número de árboles elegido.

Línea 140 · nota nueva · 28 oct 2026

```
def registrar(motor, paquete: dict, activar: bool, usuario: str) -> int:
```

Registra el paquete en model_registry; con activar=True pasa a ser el campeón.

Línea 164 · main() · nota nueva · 28 oct 2026

```
parser.add_argument("--trials", type=int, default=20, help="Trials de Optuna por horizonte (0 =...
```

CLI: –trials, –hasta, –sin-comparar, –no-activar y –horizontes.

Línea 187 · main() · nota nueva · 28 oct 2026

```
ajuste_info = ajuste.ajustar(df[df["ts"] < corte_test], FEATURES, args.trials)
```

El ajuste usa solo datos anteriores al corte de prueba.

frecuencias/inferencia/retencion.py

90 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 29 oct 2026

Retención de paquetes de modelos. Archiva, nunca borra: mueve el paquete a modelos/archivo/ y actualiza su ruta en el registro, de modo que la huella SHA-256 se sigue verificando y el modelo se puede reactivar.

Línea 14 · nota nueva · 29 oct 2026

```
RETIRADOS_A_CONSERVAR = 2
```

Se conservan el activo, los candidatos pendientes, lo que esté en activos.json y los 2 retirados más recientes por horizonte, para poder revertir.

Línea 18 · nota nueva · 29 oct 2026

```
def planificar(filas: list[dict], retirados_a_conservar: int = RETIRADOS_A_CONSERVAR) -> list[d...
```

Archiva los rechazados y los retirados más antiguos; nunca un paquete que otro registro o activos.json todavía usa.

Línea 34 · nota nueva · 29 oct 2026

```
def archivar(conn: Connection, retirados_a_conservar: int = RETIRADOS_A_CONSERVAR, simular: boo...
```

Primera ejecución: archivó los 3 paquetes rechazados (ids 35 a 37).

Línea 63 · nota nueva · 29 oct 2026

```
def archivar_fase1(simular: bool = False) -> list[str]:
```

Movió los XGBoost de la fase 1 a modelos/archivo/xgboost-fase1/; el cuaderno feature_engineering.ipynb se

actualizó a esa ruta.

Optimización de frecuencia

frecuencias/optimizacion/parametros.py

92 líneas · modificado · 4 notas

Descripción del archivo · modificado · 29 oct 2026

Parámetros de cada hora según franja y tipo de día, desde la tabla versionada.

Línea 3 · nota nueva · 28 oct 2026

```
FRANJAS = [(4, 6, "04-06"), (6, 9, "06-09"), (9, 12, "09-12"), (12, 15, "12-15"), (15, 18, "15-...
```

Franjas del análisis de flota.

Línea 22 · nota nueva · 29 oct 2026

```
"umbral_alta_demanda": 100.0,
```

Calibración de alertas: umbral de ALT-001 (100 %) y tolerancia de ALT-003 (2 buses o 15 %), con valores por defecto iguales a los de la migración 0004.

Línea 61 · nota nueva · 28 oct 2026

```
def _valor(valores: dict, *claves: str):
```

Busca primero la clave específica y después la general.

Línea 74 · parametros_hora() · nota nueva · 28 oct 2026

```
nocturno = tipo == "dom_fest" or hora >= HORA_FIN_DIURNO or hora < HORA_INICIO_DIURNO
```

Headway máximo de 15 min en domingos y festivos, antes de las 5:00 y desde las 20:00.

frecuencias/optimizacion/frecuencia.py

70 líneas · nuevo · 6 notas

Descripción del archivo · nuevo · 28 oct 2026

Recomendación de frecuencia (§10, RF-007).

Línea 8 · nota nueva · 28 oct 2026

```
def buses_necesarios(carga: float, capacidad: float, ocupacion: float) -> int:
```

buses = ⌈carga máxima / (capacidad × ocupación objetivo)⌉.

Línea 15 · recomendar() · nota nueva · 28 oct 2026

```
carga = demanda_hora * p.factor_carga
```

Carga a bordo = subidas × factor de carga (≈0,39–0,43, confirmado por segmentos).

Línea 18 · recomendar() · nota nueva · 28 oct 2026

```
por_demanda = buses_necesarios(carga_alta, p.capacidad, p.ocupacion_objetivo)
```

Se dimensiona con el límite superior del intervalo.

Línea 23 · recomendar() · nota nueva · 28 oct 2026

```
tope = math.floor(60 / p.headway_minimo)
```

Si se excede el headway mínimo, se topa y queda no viable con el motivo.

Línea 29 · recomendar() · nota nueva · 28 oct 2026

```
if despachos_en_ruta is None:
```

Vehículos: despachos en ruta más los nuevos, o ⌈ciclo / headway⌉.

Línea 61 · nota nueva · 28 oct 2026

```
def sensibilidad(demanda_hora: float, demanda_hora_alta: float, p: ParametrosHora) -> dict:
```

Escenarios con ocupación de 65, 75 y 85 % y demanda puntual o alta.

`frecuencias/optimizacion/oferta.py`

32 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 28 oct 2026

Oferta actual de la ruta a partir de los despachos.

Línea 6 · nota nueva · 28 oct 2026

```
def _despachos(viajes: pd.DataFrame, id_ruta: int) -> pd.Series:
```

Despachos no cancelados, con fecha convertida aunque la tabla llegue vacía.

Línea 11 · nota nueva · 28 oct 2026

```
def buses_actuales(viajes: pd.DataFrame, id_ruta: int, inicio_hora: pd.Timestamp) -> tuple[floa...
```

Buses observados ese día o mediana de las 4 semanas previas.

Línea 28 · nota nueva · 28 oct 2026

```
def despachos_en_ruta(viajes: pd.DataFrame, id_ruta: int, inicio_hora: pd.Timestamp, ciclo_min:...
```

Vehículos que salieron dentro de un tiempo de ciclo antes de la hora.

Alertas y notificaciones

`frecuencias/alertas/reglas.py`

180 líneas · modificado · 9 notas

Descripción del archivo · modificado · 29 oct 2026

Reglas de alerta ALT-001 a ALT-010 (§5.2). Cada alerta lleva explicación, recomendación, valores observado y esperado, umbral y clave de deduplicación.

Línea 14 · nota nueva · 28 oct 2026

```
def _alerta(codigo: str, severidad: str, id_ruta: int | None, inicio, fin, explicacion: str, re...
```

La clave de deduplicación es código:ruta:hora de la ventana.

Línea 22 · nota nueva · 29 oct 2026

```
def oferta_minima_aceptable(recomendados: float, p: ParametrosHora) -> float:
```

Calibración: ALT-003 salta solo si los buses operados quedan en o por debajo de max(recomendados - 2, recomendados × 0,85). En la demostración bajó de 274 a 252 alertas: la mayoría de los déficits son de 2 buses o más.

Línea 34 · por_recomendacion() · nota nueva · 29 oct 2026

```
if ocupacion_alta >= p.umbral_alta_demanda:
```

Calibración: ALT-001 usa la condición literal del §5.2 (la demanda proyectada supera la capacidad disponible, 100 %) en lugar del 85 %; bajó de 222 a 151.

Línea 75 · nota nueva · 28 oct 2026

```
def por_observacion(obs: dict, error_pronostico: float) -> list[dict]:
```

ALT-005 y ALT-007: demanda observada fuera del intervalo y del error tolerado.

Línea 99 · nota nueva · 28 oct 2026

```
def por_despachos(salidas: pd.Series, id_ruta: int, headway_programado: float, desde, hasta) ->...
```

ALT-006: hueco entre despachos mayor a dos headways.

Línea 120 · nota nueva · 28 oct 2026

```
def por_calidad(problemas: list[dict], execution_id: int | None) -> list[dict]:
```

ALT-008 y ALT-009 por calidad, agrupadas por ruta y día.

Línea 147 · nota nueva · 28 oct 2026

```
def por_retraso(ultima_franja, ahora, periodos_tolerados: float, horas_por_periodo: float = 24)...
```

ALT-009 por datos atrasados; ahora el ciclo con origen base la evalúa (antes estaba definida pero no se usaba).

Línea 162 · nota nueva · 29 oct 2026

```
def por_fuente_no_disponible(error: str, ahora) -> list[dict]:
```

ALT-009 crítica cuando la base del operador no responde tras los reintentos; la marca de agua no avanza.

Línea 170 · nota nueva · 28 oct 2026

```
def por_modelo(wape_movil: float | None, tolerancia: float, horizonte_min: int, model_registry_...
```

ALT-010: WAPE móvil sobre la tolerancia.

`frecuencias/alertas/notificaciones.py`

142 líneas · reescrito · 5 notas

Descripción del archivo · reescrito · 29 oct 2026

Notificaciones por correo y WhatsApp (RF-012). Sin credenciales el envío se simula (como permite la especificación); con credenciales se envía de verdad, con timeout, reintentos y circuit breaker.

Línea 22 · nota nueva · 29 oct 2026

```
class Adaptador:
```

Base común: simula si el canal no está configurado y, si lo está, envía con reintento solo ante errores transitorios.

Línea 57 · enviar_real() · nota nueva · 29 oct 2026

```
correo = EmailMessage()
```

Correo por SMTP con TLS obligatorio y certificado verificado: STARTTLS en el puerto 587 o TLS implícito en el 465. Corrección por SonarQube: antes se podía desactivar TLS (hallazgo crítico S4830).

Línea 89 · cuerpo() · nota nueva · 29 oct 2026

```
def cuerpo(self, destinatario: str, asunto: str, mensaje: str) -> dict:
```

WhatsApp Cloud API: texto libre o, con NOTIFICACION_WHATSAPP_PLANTILLA, una plantilla aprobada, necesaria para escribir fuera de la ventana de 24 h.

Línea 104 · enviar_real() · nota nueva · 29 oct 2026

```
if respuesta.status_code == 429 or respuesta.status_code >= 500:
```

Solo se reintentan 429 y 5xx; un 4xx (token inválido, número mal formado) falla de inmediato.

Línea 122 · nota nueva · 29 oct 2026

```
def notificar(conn: Connection, id_alerta, alerta: dict) -> int:
```

Registra cada intento con su estado (simulada, enviada o fallida), los intentos y el error; un fallo no interrumpe el ciclo.

Operación: predicción, recomendación y ciclo incremental

`frecuencias/operacion/pronostico.py`

47 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Ejecuta el predictor para una ruta y guarda predicciones e instantáneas de variables.

Línea 12 · nota nueva · 28 oct 2026

```
def pronosticar(predictor: Predictor, serie: pd.DataFrame, id_ruta: int, referencia: pd.Timesta...
```

Ruta inexistente como LookupError (404).

Línea 23 · nota nueva · 28 oct 2026

```
def guardar(conn: Connection, resultados: list[dict], id_ruta: int, referencia: pd.Timestamp,
```

Guarda en prediction y feature_snapshot, ambos con upsert.

frecuencias/operacion/recomendacion.py

70 líneas · modificado · 4 notas

Descripción del archivo · modificado · 29 oct 2026

Convierte las predicciones por franja en recomendaciones por hora.

Línea 13 · nota nueva · 28 oct 2026

```
def bloques(predicciones: list[dict]) -> list[list[dict]]:
```

Bloques de cuatro franjas consecutivas.

Línea 19 · nota nueva · 28 oct 2026

```
def recomendar_bloques(predicciones: list[dict], id_ruta: int, valores: dict, viajes: pd.DataFr...
```

Recomienda con la demanda puntual y la alta y evalúa las alertas.

Línea 23 · recomendar_bloques() · nota nueva · 29 oct 2026

```
if not any(r.get("servicio_habitual", True) for r in bloque):
```

Corrección: no se recomienda frecuencia en horas completamente fuera del servicio habitual.

Línea 56 · nota nueva · 28 oct 2026

```
def fila_recomendacion(rec: dict, execution_id: int | None) -> dict:
```

Fila de recommendation con parámetros y sensibilidad en JSON.

frecuencias/operacion/servicio.py

50 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 28 oct 2026

Flujo de una referencia: predicción → recomendación → alertas → notificación, en una transacción.

Línea 11 · nota nueva · 28 oct 2026

```
COLUMNAS_ALERTA = {"codigo", "severidad", "clave_deduplicacion", "id_ruta", "ventana_inicio", "...
```

Solo estas claves de la alerta van a la tabla.

Línea 17 · nota nueva · 28 oct 2026

```
def registrar_alertas(conn: Connection, lista: list[dict], execution_id: int | None) -> dict:
```

Deduplica y notifica solo las nuevas.

Línea 34 · nota nueva · 28 oct 2026

```
def ejecutar(conn: Connection, predictor: Predictor, serie: pd.DataFrame, viajes: pd.DataFrame ...
```

Lo usan la API y el ciclo incremental.

frecuencias/operacion/reentrenamiento.py

84 líneas · nuevo · 4 notas

Descripción del archivo · nuevo · 29 oct 2026

Reentrenamiento controlado campeón/retador (RF-019).

Línea 16 · nota nueva · 29 oct 2026

```
def inicio_ventana(fin: pd.Timestamp, corte_activo: pd.Timestamp, ventana_dias: int = VENTANA_D...
```

La ventana empieza después del corte del activo y deja días nuevos al retador. Corrección: antes podía empezar en el corte del activo y el retador salía idéntico.

Línea 24 · nota nueva · 28 oct 2026

```
def comparar(paquete: artefactos.Paquete, df: pd.DataFrame, inicio_ventana: pd.Timestamp) -> di...
```

Retador y activo se evalúan en días que ninguno vio.

Línea 41 · comparar() · nota nueva · 28 oct 2026

```
mejora = cand["WAPE"] < act["WAPE"] - MEJORA_MINIMA_WAPE or cand["desviacion_poisson"] < act["d...
```

Gana solo si mejora una métrica sin empeorar la otra y supera a la línea base.

Línea 49 · nota nueva · 28 oct 2026

```
def campeon_retador(motor, paquetes: dict, serie: pd.DataFrame, calendario, usuario: str = "cic...
```

Si gana se reentrena con todo, se registra y se activa; si no, queda la decisión.

frecuencias/operacion/ciclo.py

207 líneas · modificado · 11 notas

Descripción del archivo · modificado · 29 oct 2026

Ciclo incremental (§9.5, §14.2): ingesta → segmentos → predicción → recomendación → alertas → monitoreo → reentrenamiento → retención.

Línea 26 · nota nueva · 29 oct 2026

```
ERRORES_FUENTE = (OperationalError, InterfaceError, CircuitoAbierto)
```

Errores de la base del reto que generan ALT-009 y se registran sin traza.

Línea 30 · nota nueva · 29 oct 2026

```
DIAS_MINIMOS_MONITOREO = 3
```

Corrección: ALT-010 se evalúa solo con al menos 3 días observados; con un día de un modelo recién activado generó dos alertas falsas, que se descartaron. Además, antes se mezclaban el modelo anterior y el nuevo del mismo horizonte.

Línea 36 · nota nueva · 28 oct 2026

```
def extraer_local(desde: pd.Timestamp, hasta: pd.Timestamp) -> tuple[pd.DataFrame, pd.DataFrame...
```

Origen archivos: simula la llegada de datos desde datos/.

Línea 43 · nota nueva · 28 oct 2026

```
def ingerir(destino, origen: str, dias: int, id_padre: int) -> dict:
```

Lee desde la última franja guardada y registra su ejecución con marca de agua.

Línea 58 · ingerir() · nota nueva · 28 oct 2026

```
marca_agua_hasta=a_zona(hasta), detalle={"origen": origen, "ciclo": id_padre, **resumen})
```

El detalle de la ingesta conserva el enlace con su ciclo.

Línea 63 · nota nueva · 28 oct 2026

```
def monitorear(conn, paquetes: dict, serie: pd.DataFrame, hasta: pd.Timestamp, tolerancia: floa...
```

WAPE móvil de 7 días del modelo activo por horizonte.

Línea 99 · nota nueva · 29 oct 2026

```
def actualizar_segmentos(destino, origen: str, ingesta: dict, id_padre: int) -> dict:
```

Actualiza los segmentos de los días nuevos con origen base; un fallo queda registrado sin detener el ciclo.

Línea 140 · ejecutar_ciclo() · nota nueva · 28 oct 2026

```
conocidas = set(serie["ts"])
```

Las franjas conocidas se calculan una vez por ciclo.

Línea 150 · ejecutar_ciclo() · nota nueva · 28 oct 2026

```
bloques_max=1, observado=observado)
```

Se predice cada hora de 5:00 a 20:00 y se guarda la recomendación de la hora siguiente.

Línea 162 · ejecutar_ciclo() · nota nueva · 29 oct 2026

```
LATENCIA_DATOS.set((ahora - ultima) / pd.Timedelta(hours=1))
```

Latencia de los datos (métrica) y ALT-009 por retraso con origen base. Con los datos del reto, que terminan en 2025, la latencia es de ~7 700 h y la alerta es correcta.

Línea 174 · ejecutar_ciclo() · nota nueva · 29 oct 2026

```
resumen["retencion"] = retencion.archivar(conn)
```

Tras activar un modelo se aplica la retención.

`frecuencias/operacion/programador.py`

44 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Programador del ciclo con APScheduler (servicio programador de Docker Compose).

Línea 12 · nota nueva · 28 oct 2026

```
CRON_POR_DEFECTO = "15 * * * *"
```

A los 15 minutos de cada hora; se cambia con CICLO_CRON.

Línea 36 · main() · nota nueva · 28 oct 2026

```
args.reentrenar], id="ciclo", max_instances=1, coalesce=True, misfire_grace_time=900)
```

Nunca corren dos ciclos a la vez.

API y tablero

`frecuencias/api/main.py`

608 líneas · reescrito · 14 notas

Descripción del archivo · reescrito · 29 oct 2026

API v1 (§12.2) con autenticación por roles. Las notas de este archivo en el anexo de la fase 1 quedan reemplazadas por estas.

Línea 50 · nota nueva · 29 oct 2026

```
DOCUMENTACION = ENTORNO != "produccion" or os.environ.get("API_DOCS", "").lower() == "activa"
```

Swagger restringido por ambiente (§12.1): con ENTORNO=produccion no se publican /docs ni /openapi.json, salvo API_DOCS=activa.

Línea 54 · nota nueva · 28 oct 2026

```
CICLO = threading.Lock()
```

Impide lanzar un segundo ciclo mientras otro corre (409).

Línea 57 · nota nueva · 28 oct 2026

```
def _leer_historia(motor) -> tuple[pd.DataFrame, pd.DataFrame | None, str]:
```

Lee el histórico de ai_frecuencias o, sin base, la copia local.

Línea 135 · nota nueva · 29 oct 2026

```
def principal(clave: str | None = Security(esquema_clave)) -> dict:
```

Autenticación por la cabecera X-API-Key (401 si falta o no es válida). Con API_AUTENTICACION=desactivada, solo para desarrollo, todo se permite.

Línea 148 · nota nueva · 29 oct 2026

```
def requiere(rol: str):
```

Autorización por rol (403 si no alcanza).

Línea 159 · nota nueva · 29 oct 2026

```
def _usuario(quien: dict, declarado: str | None) -> str:
```

La auditoría (alertas, activación, parámetros, ejecuciones) registra el nombre de la clave, no un usuario escrito por el cliente.

Línea 215 · nota nueva · 28 oct 2026

```
def _respuesta_prediccion(r: dict, id_ruta: int, referencia: pd.Timestamp, generado: datetime,
```

Formato del §12.3, con intervalCalibrated.

Línea 241 · nota nueva · 28 oct 2026

```
def _correr(peticion: esq.PeticionPrediccion, request: Request, quien: dict) -> dict:
```

Predice con o sin guardar y agrega el tramo crítico por segmento a cada recomendación.

Línea 305 · nota nueva · 28 oct 2026

```
def _ciclo_en_segundo_plano(motor, peticion: esq.PeticionEjecucion, ejecucion: dict) -> None:
```

202 con el id y el ciclo en segundo plano; al terminar recarga el estado.

Línea 407 · nota nueva · 29 oct 2026

```
@router.get("/predictions/{prediction_id}/segments", dependencies=[Depends(LECTOR)])
```

Predicción desagregada por segmento; /segments y /segments/profile dan el catálogo y el perfil.

Línea 449 · nota nueva · 29 oct 2026

```
@router.get("/alerts/summary", dependencies=[Depends(LECTOR)])
```

Resumen de alertas con porcentaje de descartadas; va antes de /alerts/{id}.

Línea 537 · nota nueva · 28 oct 2026

```
def activar_modelo(id_modelo: int, peticion: esq.PeticionActivacion, quien: dict = Depends(ADMI...
```

Solo administrador; verifica la integridad antes de activar.

Línea 572 · nota nueva · 28 oct 2026

```
def exportar(recurso: Literal["predictions", "recommendations", "alerts", "executions", "data-q...
```

CSV o XLSX hasta 100 000 filas, con fechas en hora local.

Línea 602 · nota nueva · 28 oct 2026

```
app.mount("/tablero", StaticFiles(directory=CARPETA_TABLERO, html=True), name="tablero")
```

El tablero se sirve en /tablero.

frecuencias/api/esquemas.py

102 líneas · reescrito · 4 notas

Descripción del archivo · reescrito · 29 oct 2026

Contratos de entrada y salida de la API.

Línea 28 · nota nueva · 28 oct 2026

```
class PrediccionRespuesta(BaseModel):
```

Respuesta con los nombres del §12.3.

Línea 38 · PrediccionRespuesta · nota nueva · 29 oct 2026

```
intervalCalibrated: bool = Field(True, description="False en franjas sin servicio habitual: el ...
```

False en franjas sin servicio habitual.

Línea 75 · nota nueva · 29 oct 2026

```
Usuario = Annotated[str | None, Field(min_length=1, description="Solo si la autenticacion esta ...
```

El usuario del cuerpo es opcional: con autenticación se usa el de la clave.

Línea 86 · nota nueva · 28 oct 2026

```
class PeticionEjecucion(BaseModel):
```

Origen, de 1 a 31 días y política de reentrenamiento.

frecuencias/api/errores.py

45 líneas · modificado · 2 notas

Descripción del archivo · modificado · 29 oct 2026

Errores uniformes (§12.1).

Línea 9 · nota nueva · 29 oct 2026

```
CODIGOS_HTTP = {400: "solicitud_invalida", 401: "no_autenticado", 403: "sin_permiso", 404: "no_...
```

Se agregaron no_autenticado (401) y sin_permiso (403).

Línea 19 · nota nueva · 28 oct 2026

```
def registrar_manejadores(app: FastAPI) -> None:
```

LookupError → 404, ValueError → 400, validación → 422, base caída → 503 y cualquier otro → 500 con el correlation_id.

frecuencias/tablero/index.html

150 líneas · modificado · 4 notas

Descripción del archivo · modificado · 29 oct 2026

Tablero operativo sin dependencias externas: predicción, alertas, ejecuciones, modelos, parámetros y exportación.

Línea 7 · nota nueva · 29 oct 2026

```
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 ...
```

Ícono embebido: evita el 404 de /favicon.ico que la prueba detectó en la consola.

Línea 23 · nota nueva · 29 oct 2026

```
<form id="form-clave" class="clave" autocomplete="off">
```

Campo para la clave de acceso; se guarda solo en la sesión del navegador.

Línea 30 · nota nueva · 28 oct 2026

`<nav class="pestanas" role="tablist" aria-label="Secciones">`

Pestañas accesibles con flechas, Inicio y Fin.

Línea 64 · nota nueva · 29 oct 2026

`<div class="tabla-envoltura"><table id="pred-tabla"><thead><tr><th scope="col">Franja</th></tr>...`

Encabezados iniciales en las tablas (hallazgo de SonarQube S5256); el JavaScript los reemplaza al cargar.

frecuencias/tablero/app.js

452 líneas · modificado · 7 notas

Descripción del archivo · modificado · 29 oct 2026

Lógica del tablero.

Línea 29 · nota nueva · 29 oct 2026

`function cabeceras(extra = {}) {`

Toda petición lleva la clave; 401 y 403 se explican en pantalla.

Línea 34 · nota nueva · 28 oct 2026

`function escapar(valor) {`

Todo texto de la API se escapa antes de insertarlo.

Línea 38 · nota nueva · 28 oct 2026

`function fecha(valor) {`

Fechas con zona en hora de Bogotá; expresión regular agrupada (SonarQube S5850).

Línea 158 · nota nueva · 29 oct 2026

`if (ultima) $("pred-referencia").value = ‘${String(ultima).slice(0, 10)}T06:00‘;`

La referencia por defecto es las 6:00 del último día con datos, no la última franja nocturna.

Línea 161 · nota nueva · 28 oct 2026

`function grafico(predicciones) {`

Gráfico SVG propio con banda del intervalo, predicción, observado y línea base.

Línea 188 · nota nueva · 29 oct 2026

`function tramoCritico(t) {`

Muestra el segmento más cargado a esa hora y su ocupación.

Línea 402 · nota nueva · 29 oct 2026

`async function exportar(evento) {`

La exportación usa fetch con la clave y descarga el archivo.

frecuencias/tablero/estilos.css

152 líneas · modificado · 0 notas

Descripción del archivo · modificado · 29 oct 2026

Tema claro y oscuro, foco visible y diseño adaptable a 375 px. Contraste mínimo medido: 5,69:1 (WCAG AA exige 4,5:1).

Migraciones de la base ai_frecuencias

migrations/versions/0002_historico_calidad_notificaciones.py

150 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Histórico, calidad, notificaciones e instantáneas de variables.

Línea 37 · nota nueva · 28 oct 2026

```
def _crear_historicos() -> None:
```

demand_history y trip_history.

Línea 81 · nota nueva · 28 oct 2026

```
def _crear_calidad_y_trazabilidad() -> None:
```

data_quality_issue, notification y feature_snapshot.

migrations/versions/0003_parametros_operacion.py

127 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 28 oct 2026

Parámetros del análisis de flota 2025 (capacidad, factores de carga, tiempos de ciclo, flota y headways), cargados como no validados.

Línea 10 · nota nueva · 28 oct 2026

```
CREADOR = "semilla analisis de flota 2025"
```

Autor de la versión.

Línea 124 · nota nueva · 28 oct 2026

```
def downgrade() -> None:
```

Revertir borra solo las filas de esta semilla.

migrations/versions/0004_calibracion_alertas.py

37 líneas · nuevo · 1 nota

Descripción del archivo · nuevo · 29 oct 2026

Umbrales de calibración de alertas como parámetros versionados (§5.1: modificables sin recompilar).

Línea 11 · nota nueva · 29 oct 2026

```
PARAMETROS = [
```

umbral_alta_demanda 100 %, tolerancia_frecuencia_buses 2 y tolerancia_frecuencia_pct 15 %, no validados hasta que operación los confirme.

migrations/versions/0005_claves_api.py

36 líneas · nuevo · 2 notas

Descripción del archivo · nuevo · 29 oct 2026

Tabla api_key.

Línea 27 · upgrade() · nota nueva · 29 oct 2026

```
sa.CheckConstraint("activa OR revocada_en IS NOT NULL", name="ck_api_key_revocada"),
```

Una clave inactiva debe tener fecha de revocación.

Línea 31 · upgrade() · nota nueva · 29 oct 2026

```
op.create_index("uq_api_key_nombre_activa", "api_key", ["nombre"], unique=True, schema=ESQUEMA,
```

Nombre único solo entre las claves activas: se puede reutilizar tras revocar.

migrations/versions/0006_segmentos.py

93 líneas · nuevo · 3 notas

Descripción del archivo · nuevo · 29 oct 2026

Segmentos: route_segment con geometría e índice GiST (§8.1), segment_observation y segment_profile.

Línea 19 · upgrade() · nota nueva · 29 oct 2026

```
op.execute("CREATE EXTENSION IF NOT EXISTS postgis")
```

La imagen postgis ya la trae; la migración la declara.

Línea 34 · upgrade() · nota nueva · 29 oct 2026

```
op.execute(f"CREATE INDEX ix_segmento_geom ON {ESQUEMA}.route_segment USING gist (geom)")
```

Índice espacial para la búsqueda del segmento más cercano.

Línea 46 · upgrade() · nota nueva · 29 oct 2026

```
sa.Column("pasajeros", sa.BigInteger, nullable=False),
```

BIGINT porque la fórmula del operador produce valores enormes (hasta 137 818).

Pruebas y herramientas de verificación

tests/test_seguridad.py

53 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Huella, generación, jerarquía de roles, crear, validar y revocar.

tests/test_resiliencia.py

77 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Circuit breaker (apertura, semiapertura y cierre), reintento exponencial y saneo de errores.

tests/test_notificaciones.py

100 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Simulación sin credenciales, SMTP con STARTTLS y con TLS implícito, reintentos, WhatsApp con texto y plantilla, rechazo 4xx sin reintento y registro de fallos.

tests/test_retencion.py

90 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Plan de retención y archivo de paquetes y de la fase 1.

tests/test_segmentos.py

82 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Perfil con ceros implícitos y cuotas, tipos de día con festivos y asignación espacial con carga balanceada sobre una ruta sintética.

tests/test_ciclo.py

50 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Mínimo de días del monitoreo, solo modelos activos y omisión de segmentos.

tests/test_reentrenamiento.py

24 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 28 oct 2026

Ventana del campeón/retador.

tests/test_optimizacion.py

127 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Fórmulas, parámetros, oferta y horas sin servicio.

tests/test_alertas.py

113 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Reglas ALT y calibración de ALT-001 y ALT-003.

tests/test_artefactos.py

134 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Paquetes, integridad, predictor e intervalos fuera de servicio.

tests/test_calidad.py

68 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 28 oct 2026

Un caso por regla CAL.

tests/test_limpieza.py

34 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Ningún archivo de código tiene comentarios ni docstrings.

tests/test_api.py

151 líneas · reescrito · 0 notas

Descripción del archivo · reescrito · 29 oct 2026

API con claves reales por rol: 401, 403, predicción, segmentos, resumen de alertas, exportación y métricas.

pruebas/e2e_tablero.py

168 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Prueba del tablero en Microsoft Edge con Playwright: flujo completo, teclado, contraste WCAG en ambos temas, vista móvil y errores de consola.

pruebas/carga_api.py

102 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Prueba de carga con usuarios concurrentes (RNF-002).

pruebas/volumen_ingesta.py

93 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Prueba de volumen de la ingesta (RNF-003) en una base temporal que se crea y elimina.

pruebas/explain_fuente.py

63 líneas · nuevo · 1 nota

Descripción del archivo · nuevo · 29 oct 2026

EXPLAIN (ANALYZE, BUFFERS) de las consultas incrementales a la base del reto (§8.3).

Línea 15 · nota nueva · 29 oct 2026

```
def nodos(plan: dict) -> list[dict]:
```

Recorrido iterativo del plan (antes recursivo: hallazgo bloqueante S2190 de SonarQube).

pruebas/humo_docker.py

70 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Prueba de humo de la API desplegada con Docker Compose.

pruebas/informe_sonar.py

81 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Resumen del análisis de SonarQube por la API del servidor.

Contenedores, CI y configuración

`Dockerfile`

28 líneas · modificado · 1 nota

Descripción del archivo · modificado · 29 oct 2026

Imagen de la API.

Línea 28 · nota nueva · 29 oct 2026

```
CMD ["sh", "-c", "alembic upgrade head && exec uvicorn frecuencias.api.main:app --host 0.0.0.0 ...
```

Uvicorn cerraba conexiones inactivas a los 5 s; un cliente que consultaba cada 5 s reutilizaba una conexión cerrada (condición de carrera).

`docker-compose.yml`

90 líneas · modificado · 4 notas

Descripción del archivo · modificado · 29 oct 2026

API, base y programador con un ancla común de variables.

Línea 3 · nota nueva · 28 oct 2026

```
x-base-ia: &base-ia
```

Variables explícitas, nunca el .env completo.

Línea 13 · nota nueva · 28 oct 2026

```
SOBUSA_DB_PASSWORD: ${SOBUSA_DB_PASSWORD:-${PGPASSWORD:-}}
```

Toma SOBUSA_DB_PASSWORD o PGPASSWORD del equipo.

Línea 17 · nota nueva · 29 oct 2026

```
API_AUTENTICACION: ${API_AUTENTICACION:-activa}
```

Autenticación, ambiente, SMTP, WhatsApp y segmentos.

Línea 72 · nota nueva · 28 oct 2026

```
programador:
```

Servicio opcional con –profile programador.

`.env.example`

40 líneas · modificado · 0 notas

Descripción del archivo · modificado · 29 oct 2026

Plantilla sin secretos con todas las variables.

`requirements.txt`

20 líneas · modificado · 0 notas

Descripción del archivo · modificado · 29 oct 2026

Se agregó httpx (adaptador de WhatsApp).

`requirements-dev.txt`

7 líneas · modificado · 0 notas

Descripción del archivo · modificado · 29 oct 2026

Se agregaron Playwright y psutil para pruebas/.

.github/workflows/ci.yml

62 líneas · modificado · 3 notas

Descripción del archivo · modificado · 28 oct 2026

Integración continua.

Línea 42 · nota nueva · 28 oct 2026

```
- run: python -m alembic downgrade base && python -m alembic upgrade head
```

Migraciones de ida y vuelta.

Línea 43 · nota nueva · 28 oct 2026

```
- run: python -m pytest -m "not lenta" --cov=frecuencias --cov-report=xml --cov-report=term --j...
```

Pruebas con cobertura y reporte JUnit.

Línea 51 · nota nueva · 28 oct 2026

```
- name: SonarQube
```

Análisis si hay secretos configurados.

sonar-project.properties

10 líneas · modificado · 1 nota

Descripción del archivo · modificado · 29 oct 2026

Configuración de SonarQube.

Línea 9 · nota nueva · 29 oct 2026

```
sonar.coverage.exclusions=migrations/**,pruebas/**,frecuencias/tablero/**
```

Migraciones, scripts de pruebas/ y JavaScript del tablero fuera de la cobertura.

.coveragerc

6 líneas · nuevo · 0 notas

Descripción del archivo · nuevo · 29 oct 2026

Rutas relativas en coverage.xml para que SonarQube asocie la cobertura.

pytest.ini

9 líneas · modificado · 0 notas

Descripción del archivo · modificado · 29 oct 2026

Ignora la advertencia esperada del tipo geometry.

Anexo

Documento de la fase 1 (15 de octubre de 2026), sin cambios y con su paginación original

Comentarios del código

SOBUSA Data Challenge 2026 · Predicción de demanda de pasajeros · actualizado el 15 de octubre de 2026

El código del proyecto no tiene comentarios ni docstrings. Este documento reúne todo lo que antes explicaban, junto con las explicaciones de cada cambio posterior.

Empieza con el registro de cambios, del más reciente al más antiguo. Después vienen las notas de cada archivo: cada una indica la **línea del código actual** a la que acompaña, la función donde está y, en gris, esa línea de código.

Las notas sin marca conservan el texto original del comentario. La marca **actualizada** señala un comentario original cuyo texto se ajustó porque nombraba archivos o comandos que cambiaron. La marca **nota nueva** señala una explicación escrita al agregar o cambiar código, con su fecha.

Archivo	Líneas	Notas del código original	Notas nuevas
`frecuencias/config.py`	23	11	3
`frecuencias/db.py`	67	1	6
`frecuencias/ingesta/consultas.py`	37	1	0
`frecuencias/ingesta/extraccion.py`	74	2	1
`frecuencias/analitica/calidad.py`	20	3	0
`frecuencias/analitica/carga.py`	66	4	0
`frecuencias/analitica/serie.py`	28	1	0
`frecuencias/analitica/features.py`	101	15	1
`frecuencias/inferencia/evaluacion.py`	24	1	0
`frecuencias/inferencia/entrenamiento.py`	137	4	3
`frecuencias/inferencia/modelos.py`	32	1	1
`frecuencias/api/esquemas.py`	37	0	0
`frecuencias/api/main.py`	188	6	4
`migrations/env.py`	24	0	2
`migrations/versions/0001_esquema_inicial.py`	294	0	8
`migrations/script.py.mako`	15	0	0
`alembic.ini`	38	0	1
`Dockerfile`	28	0	3
`docker-compose.yml`	42	0	5
`.env.example`	13	0	0
`.gitignore`	4	0	0
`.dockerignore`	16	0	0

Archivo	Líneas	Notas del código original	Notas nuevas
`requirements.txt`	12	0	0
`requirements-dev.txt`	3	0	0
Total	**1323**	**50**	**38**

Registro de cambios

15 de octubre de 2026 · Fase 1: base del producto

- El código se reorganizó en el paquete `frecuencias`, separado por responsabilidad como pide el §7.1 de la especificación: ingesta, analítica, inferencia y exposición (API). La lógica no cambió: los modelos reentrenados con la nueva estructura son idénticos bit a bit a los anteriores.
- Se creó la base de datos propia de la solución con Alembic: siete tablas en el esquema `ai_frecuencias` (§8.2), con sus restricciones e índices, y los umbrales del §5.1 como datos semilla.
- El proyecto se levanta con Docker Compose (RNF-008): PostgreSQL 14 con PostGIS y la API, que aplica las migraciones al arrancar.
- La API quedó bajo la ruta base `/api/v1` (§12.1) e informa en `/api/v1/salud` el estado de la base `ai_frecuencias`.
- La configuración sale de variables de entorno y el repositorio deja de guardar el archivo `.env` (§7.1 y §19).

Nueva estructura del código

Módulo	Responsabilidad	Viene de
`frecuencias/config.py`	Constantes y rutas de carpetas	modelo_demanda.py y api_demanda.py
`frecuencias/db.py`	Conexiones a la base del reto y a ai_frecuencias	_crear_engine de modelo_demanda.py, más código nuevo
`frecuencias/ingesta/consultas.py`	Consultas SQL de solo lectura	modelo_demanda.py y extraer_datos.py
`frecuencias/ingesta/extraccion.py`	Copia local de los datos en datos/	extraer_datos.py
`frecuencias/analitica/calidad.py`	Reglas de calidad	modelo_demanda.py
`frecuencias/analitica/carga.py`	Carga de la demanda y del calendario	modelo_demanda.py
`frecuencias/analitica/serie.py`	Grilla regular de 15 minutos	modelo_demanda.py
`frecuencias/analitica/features.py`	Las 22 variables del modelo	modelo_demanda.py
`frecuencias/inferencia/evaluacion.py`	Métricas y tipo de día	modelo_demanda.py
`frecuencias/inferencia/entrenamiento.py`	Entrenamiento, evaluación y modelo final	modelo_demanda.py
`frecuencias/inferencia/modelos.py`	Carga y validación de los modelos	api_demanda.py
`frecuencias/api/esquemas.py`	Esquemas de las respuestas	api_demanda.py
`frecuencias/api/main.py`	Servicio FastAPI	api_demanda.py
`migrations/` y `alembic.ini`	Migraciones de ai_frecuencias	nuevo
`Dockerfile` y `docker-compose.yml`	Contenedores	nuevo

Comandos

Tarea	Antes	Ahora
Extraer los datos	`python extraer_datos.py`	`python -m frecuencias.ingesta.extraccion`
Entrenar	`python modelo_demanda.py`	`python -m frecuencias.inferencia.entrenamiento`
API en el equipo	`uvicorn api_demanda:app`	`uvicorn frecuencias.api.main:app`
Endpoints	`/salud, /modelos, /rutas, /prediccion/{id_ruta}`	`/api/v1/salud, /api/v1/modelos, /api/v1/rutas, /api/v1/prediccion/{id_ruta}`
Crear o actualizar ai_frecuencias	—	`python -m alembic upgrade head`
Ver el DDL sin conectarse	—	`python -m alembic upgrade head --sql`
Levantar todo con Docker	—	`docker compose up -d --build`

Secciones del código original

Los títulos de sección que separaban el código original se convirtieron en módulos. Su texto se conserva aquí.

Sección (texto original)	Archivo original	Ahora en
1. CARGA Y LIMPIEZA DE DATOS	modelo_demanda.py	analitica/carga.py, analitica/calidad.py y db.py
2. SERIE REGULAR DE 15 MINUTOS POR RUTA (+ validacion basica, seccion 6.3)	modelo_demanda.py	analitica/serie.py
3. VARIABLES POR HORIZONTE (sin fuga temporal: todo usa datos de t - h o antes)	modelo_demanda.py	analitica/features.py; la partición pasó a inferencia/entrenamiento.py y el tipo de día a inferencia/evaluacion.py
4. METRICAS (seccion 9.3)	modelo_demanda.py	inferencia/evaluacion.py; nuevo_modelo pasó a inferencia/entrenamiento.py
MAIN	modelo_demanda.py	función main de inferencia/entrenamiento.py
CARGA AL ARRANCAR	api_demanda.py	inferencia/modelos.py (cargar_modelos) y la función lifespan de api/main.py
ESQUEMAS DE RESPUESTA	api_demanda.py	api/esquemas.py
PREDICCION	api_demanda.py	funciones auxiliares de api/main.py
ENDPOINTS	api_demanda.py	api/main.py, bajo /api/v1

Base de datos ai_frecuencias

Migración `0001_esquema_inicial`. Las tablas usan los nombres del §8.2 y las columnas están en español, como el resto del código. Alembic guarda su tabla de versiones (`alembic_version`) dentro del mismo esquema, de modo que nada se escribe en `public`. Los identificadores de ruta, viaje y vehículo no tienen llave foránea hacia las tablas operativas porque esas tablas viven en la base del reto, que es de solo lectura.

Tabla	Para qué sirve	Reglas principales
`operational_parameter`	Umbrales configurables y versionados (§5.1)	Una sola versión vigente por clave y ruta. Ocho valores semilla, marcados como no validados
`model_registry`	Registro de modelos (§9.2, RF-018, RF-019)	Huella SHA-256 obligatoria. Un solo modelo activo por horizonte
`execution_log`	Bitácora de ejecuciones y marca de agua (RF-013)	Tipos y estados cerrados. correlation_id automático
`prediction`	Predicciones con intervalo (RF-006)	Sin duplicados por modelo, ruta, sentido, segmento, franja y horizonte. Intervalo y franja coherentes
`recommendation`	Headway, buses por hora y vehículos (RF-007)	Si no es viable, el motivo es obligatorio
`alert`	Alertas ALT-001 a ALT-010 (RF-010, §5.3)	Catálogo y severidades cerrados. No puede haber dos alertas abiertas con la misma clave
`alert_history`	Estados y comentarios de cada alerta (RF-011)	Se borra en cascada con la alerta

Umbral semilla	Valor	Unidad
`ocupacion_objetivo_min` / `ocupacion_objetivo_max`	65 / 85	%
`ocupacion_critica`	90	%
`baja_utilizacion`	30	%
`espera_maxima`	5	minutos
`headway_minimo`	sin valor: lo define la operación	minutos
`error_pronostico`	15	%
`datos_atrasados`	2	periodos de ingesta

Docker y configuración

Servicio	Imagen	Puerto en el equipo	Detalle
`db`	postgis/postgis:14-3.5 (PostgreSQL 14.18 con PostGIS 3.5.2)	5433	Datos persistentes en el volumen datos_ai
`api`	Construida desde el Dockerfile (Python 3.12, 2,04 GB)	8000	Aplica las migraciones al arrancar. Monta datos/ y modelos/ en solo lectura y recibe solo las variables de ai_frecuencias

Variable	Para qué	Detalle
`SOBUSA_DB_HOST, _PORT, _USER, _NAME, _SSLMODE`	Conexión de solo lectura a la base del reto	Valores entregados por Extreme Technologies; no van en el repositorio
`PGPASSWORD` o `SOBUSA_DB_PASSWORD`	Contraseña de la base del reto	Solo en el entorno del equipo, nunca en archivos
`AI_DB_HOST, _PORT, _NAME, _USER, _PASSWORD`	Conexión a ai_frecuencias	Fuera de Docker: localhost y 5433. Dentro de Docker: db y 5432

Variable	Para qué	Detalle
API_PORT	Puerto de la API en el equipo	8000 por defecto

Cómo levantar el proyecto

1. Copiar `.env.example` a `.env` y completar los valores. `AI_DB_PASSWORD` es una contraseña propia para la base local.
2. Levantar todo: `docker compose up -d --build`. La documentación de la API queda en `http://localhost:8000/docs` y la base en el puerto 5433 del equipo.
3. Trabajar fuera de Docker: instalar las dependencias con `pip install -r requirements-dev.txt`; con la base de Docker levantada, aplicar las migraciones con `python -m alembic upgrade head` y arrancar la API con `uvicorn frecuencias.api.main:app`.
4. La ingesta y el entrenamiento se corren desde el equipo, porque dentro del contenedor `datos/` y `modelos/` están en solo lectura: `python -m frecuencias.ingesta.extraccion` y `python -m frecuencias.inferencia.entrenamiento`.
5. Detener sin perder la base: `docker compose stop`. `docker compose down -v` borra también el volumen con los datos de ai_frecuencias.

Cambios en git y en los notebooks

- `.env` y las carpetas `__pycache__` dejaron de versionarse (siguen en el disco). Se agregaron `.gitignore` y la plantilla `.env.example`.
- Se eliminaron `modelo_demanda.py`, `extraer_datos.py` y `api_demanda.py`; su código vive ahora en el paquete `frecuencias`.
- `requirements.txt` quedó con versiones fijas y con `alembic`; lo que solo usan los notebooks pasó a `requirements-dev.txt`.
- En `eda.ipynb` y `feature_engineering.ipynb` se cambió `import modelo_demanda as md` por imports directos desde `frecuencias`, y las menciones a archivos antiguos en su texto. No se tocaron sus salidas.
- La función `_dias_en_rangos` pasó a llamarse `dias_en_rangos` porque la usa `eda.ipynb`, y `_crear_engine` pasó a `motor_fuente`. Los mensajes de error ahora muestran los comandos nuevos.

Verificación

- **Código:** de 39 funciones y bloques del código original, 30 quedaron idénticos. Los otros 9 difieren solo en lo previsto: mensajes que nombraban archivos que ya no existen, el prefijo /api/v1, el estado de la base en /api/v1/salud y la consulta de rutas convertida en constante.
- **Ingesta:** la extracción corrió en 17 s contra la base del reto y produjo datos idénticos a los anteriores. viajes.parquet coincide al ordenarlo, porque su consulta no fija el orden de las filas.
- **Entrenamiento:** corrió en 136 s y produjo los tres modelos idénticos bit a bit a los anteriores (misma huella SHA-256), con las mismas métricas.
- **Base:** la migración se aplicó en PostgreSQL 14.18 con PostGIS 3.5.2, y bajarla y volver a subirla funciona. Pasaron 22 pruebas de las reglas: duplicados rechazados, catálogos cerrados, un solo modelo activo por horizonte, deduplicación de alertas abiertas, versionado de umbrales y borrado en cascada del historial.
- **Docker:** la API arrancó en 6 s, aplicó las migraciones y respondió en /api/v1/salud (base en la revisión 0001), /api/v1/rutas y /api/v1/prediccion/16. La ruta antigua /salud responde 404. Sin base configurada, la API sigue funcionando e informa no_configurada.
- **Notebooks:** la primera celda de ambos corre sin errores con los imports nuevos.

- **Sin comentarios:** se revisaron automáticamente 20 archivos Python y 9 de configuración; ninguno tiene comentarios ni docstrings.

Pendiente u observado

- `.env` dejó de versionarse, pero sigue en el historial de git y en el repositorio remoto, con el host y el usuario de la base del reto (no la contraseña). Conviene que el repositorio sea privado o limpiar el historial.
- La imagen de la API pesa 2,04 GB, sobre todo por la versión de XGBoost para Linux, que trae librerías de GPU. El paquete `xgboost-cpu` la reduciría.
- `datos/` y `modelos/` siguen versionados en git.
- Los umbrales semilla están marcados como no validados hasta que los confirme el área operativa.
- Todavía no hay pruebas automatizadas; corresponden a la fase 6.

11 de octubre de 2026

- El modelo final de cada horizonte se reentrena con todas las franjas (entrenamiento, validación y prueba) usando el número de árboles que eligió la parada temprana. Es el que se guarda y sirve la API; las métricas reportadas son las del modelo evaluado antes de ese paso.
- El tope de árboles subió de 1000 a 3000: con 1000 los modelos llegaban al límite. La parada temprana elige 1809, 2306 y 2006 árboles para 30, 60 y 120 minutos.
- Los modelos se guardan siempre en la carpeta modelos/ del proyecto, la misma que lee la API.

3 de octubre de 2026

- Se retiraron todos los comentarios y docstrings del código y se trasladaron a este documento.

Paquete frecuencias

frecuencias/config.py

23 líneas · 11 notas del código original · 3 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Constantes del proyecto en un solo lugar: carpetas de datos y modelos, paso de 15 minutos, horizontes, versión de las variables, umbrales de las reglas de calidad, identificadores de estacionalidades y nombre del esquema de la base de IA. Antes estaban repartidas entre modelo_demanda.py y api_demanda.py.

Línea 3 · nota nueva · 15 oct 2026

```
RAIZ = Path(__file__).resolve().parents[1]
```

Carpeta raíz del proyecto (la que contiene frecuencias/). De ella cuelgan datos/ y modelos/, así que las rutas funcionan igual desde cualquier carpeta de trabajo y dentro del contenedor, donde la raíz es /app.

Línea 6 · nota nueva · 11 oct 2026 · actualizada · 15 oct 2026

```
CARPETA_MODELOS = RAIZ / "modelos"
```

Carpeta de los modelos, en la raíz del proyecto. Es la misma que lee frecuencias/inferencia/modelos.py, sin importar desde qué carpeta se ejecute el script.

Línea 9

```
PASO_MIN = 15
```

franjas de 15 minutos

Línea 10

```
PASOS_DIA = 24 * 60 // PASO_MIN
```

96 franjas por dia

Línea 11

```
PASOS_SEMANA = 7 * PASOS_DIA
```

672 franjas por semana

Línea 12

```
PASOS_ANIO = 52 * PASOS_SEMANA
```

364 dias: mismo dia de la semana un anio antes

Línea 13

```
HORIZONTES_MIN = [30, 60, 120]
```

RF-005

Línea 16

```
MIN_FRACCION_LECTURAS_DIA = 0.05
```

Reglas de calidad (ver eda.ipynb, seccion 2)

Línea 16

```
MIN_FRACCION_LECTURAS_DIA = 0.05
```

dia con < 5% de las lecturas habituales -> sin datos

Línea 17

```
MAX_SUBIDAS_LECTURA = 90
```

una lectura (~1 min) no puede superar la capacidad del bus

Línea 19

```
ID_PRECARNAVAL = 7
```

estacionalidades: Precarnaval y Carnaval de Barranquilla

Línea 20

```
ID_CARNAVAL_CENTRAL = 8
```

estacionalidades: Carnaval - dias centrales

Línea 21

```
ID_SEMANA_SANTA = 10
```

estacionalidades: Semana Santa

Línea 23 · nota nueva · 15 oct 2026

```
ESQUEMA_IA = "ai_frecuencias"
```

Esquema propio de la solución en PostgreSQL (§8.1): todas las tablas nuevas viven aquí y nunca en el esquema operacional.

---

frecuencias/db.py

67 líneas · 1 nota del código original · 6 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Conexiones a las dos bases de datos: la del reto (fuente, solo lectura) y la propia de la solución (destino, esquema ai_frecuencias). Toma la configuración de las variables de entorno y, si existe, del archivo .env de la raíz del proyecto.

Línea 11 · nota nueva · 15 oct 2026

```
load_dotenv(RAIZ / ".env")
```

Carga .env sin sobrescribir variables ya definidas: dentro de Docker mandan las variables del contenedor.

Línea 15 · nota nueva · 15 oct 2026

```
VERSION_ESQUEMA_IA = Table("alembic_version", MetaData(schema=ESQUEMA_IA), Column("version_nu…
```

Tabla de versión que Alembic mantiene dentro de ai_frecuencias. Permite saber qué migración tiene aplicada la base.

Línea 18 · motor_fuente() · nota nueva · 15 oct 2026

```
def motor_fuente() -> Engine:
```

Antes _crear_engine() en modelo_demanda.py; misma lógica, con los imports al inicio del módulo.

Línea 33 · motor_fuente()

```
return create_engine(
```

Solo lectura + timeout de 5 minutos por consulta (seccion 11.3)

Línea 42 · url_destino() · nota nueva · 15 oct 2026

```
def url_destino() -> URL | None:
```

Devuelve None si AI_DB_HOST no está definida: la API funciona sin la base de IA (por ejemplo, fuera de Docker) y lo informa en /api/v1/salud.

Línea 55 · motor_destino() · nota nueva · 15 oct 2026

```
def motor_destino() -> Engine:
```

pool_pre_ping descarta conexiones caídas y statement_timeout corta a los 30 s cualquier consulta colgada (§11.3: timeout configurado).

Línea 65 · revision_esquema_ia() · nota nueva · 15 oct 2026

```
def revision_esquema_ia(motor: Engine) -> str | None:
```

Devuelve la revisión de Alembic aplicada en la base; la usa /api/v1/salud.

---

frecuencias/ingesta/consultas.py

37 líneas · 1 nota del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Consultas SQL de solo lectura sobre la base del reto. QUERY_ESTACIONALIDADES y QUERY_NOVEDADES venían de modelo_demanda.py; QUERY_VIAJES y QUERY_CONTEO, de extraer_datos.py; QUERY_RUTAS estaba escrita dentro de la extracción y ahora es una constante con el mismo texto.

Línea 8

```
QUERY_CONTEO = """
```

Subidas por la hora real de cada lectura del sensor. Las lecturas con cantidad negativa (reinicios del contador) no suman: se cuentan aparte.

---

frecuencias/ingesta/extraccion.py

74 líneas · 2 notas del código original · 1 nota nueva

> **Descripción del archivo · docstring original de extraer_datos.py · actualizada · 15 oct 2026**
>
> Extraccion de TODO el historico disponible - SOBUSA Data Challenge 2026.
>
> timbradas_segmentos solo cubre 2023-01-01 a 2023-02-27. La base guarda mucho
> mas: los sensores de pasajeros (conteo_pasajeros_destino, ~176 millones de
> lecturas) y la programacion de viajes (progvehiculos) de 2023 a 2025.
>
> Este script arma una copia local en datos/ para que el EDA y el entrenamiento
> no dependan de consultas pesadas:
> - viajes.parquet un registro por viaje programado (progvehiculos)
> - demanda_15min.parquet subidas y bajadas por ruta y franja de 15 min,
> desde las lecturas de los sensores (hora real de
> cada subida, no la hora de salida del bus)
> - estacionalidades.parquet, novedades.parquet calendario
> - rutas.parquet nombre de cada ruta
>
> conteo_pasajeros_destino no tiene indice por fecha, solo por idprogramacion:
> por eso se consulta mes a mes con la lista de viajes del mes, agregando en el
> servidor. Cada mes se guarda aparte (datos/conteo_mensual/), asi que si se
> corta se puede volver a correr y retoma donde iba.
>
> Uso: python -m frecuencias.ingesta.extraccion (tarda del orden de 15-30 minuto)

Línea 16 · nota nueva · 15 oct 2026

```
CARPETA_MES = CARPETA_DATOS / "conteo_mensual"
```

Un archivo por mes en datos/conteo_mensual/; si la extracción se corta, al volver a correrla retoma los meses que faltan.

Línea 19 · _consultar_conteo() · docstring

```
def _consultar_conteo(conn, ids: list) -> pd.DataFrame:
```

Si una consulta excede el timeout, parte la lista en dos y reintenta.

Línea 61 · main()

```
partes = pd.concat([pd.read_parquet(a) for a in sorted(CARPETA_MES.glob("*.parquet"))])
```

Un viaje que cruza la medianoche de fin de mes deja la misma franja en dos archivos: se vuelve a agregar. (nro_viajes/vehiculos pueden sobrecontar 1 en esas pocas franjas.)

---

frecuencias/analitica/calidad.py

20 líneas · 3 notas del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Reglas de calidad encontradas en el EDA, aplicadas a la demanda por franja (antes en modelo_demanda.py).
> Sus umbrales están en frecuencias/config.py.

Línea 6 · limpiar_demanda() · docstring

```
def limpiar_demanda(df: pd.DataFrame, verbose: bool = True) -> pd.DataFrame:
```

Reglas de validez (seccion 6.3) encontradas en el EDA.

Línea 10 · limpiar_demanda()

```
lect_dia = df.groupby(["id_ruta", "fecha"])["lecturas"].transform("sum")
```

1. Dias sin lecturas del sensor: el ultimo dia de cada mes la tabla de sensores esta casi vacia aunque los viajes si se hicieron.

Línea 14 · limpiar_demanda()

```
imposibles = df["max_subida_lectura"] > MAX_SUBIDAS_LECTURA
```

2. Lecturas imposibles (saltos del contador): se descarta la franja entera

---

frecuencias/analitica/carga.py

66 líneas · 4 notas del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Carga de la demanda local (o de datos sintéticos si no existe) y del calendario de festivos, temporadas y eventos (antes en modelo_demanda.py).

Línea 20 · cargar_calendario() · docstring · actualizada · 15 oct 2026

```
def cargar_calendario():
```

Festivos, temporadas y eventos: se conocen de antemano, asi que usarlos como variables no es fuga temporal. Se leen de datos/ (frecuencias/ingesta/extraccion.py); si no estan, de la base; sin ninguna de las dos, calendario vacio.

Línea 37 · _generar_datos_sinteticos() · docstring

```
def _generar_datos_sinteticos() -> pd.DataFrame:
```

Misma forma que los datos reales: id_ruta, ts, subidas, bajadas, nro_viajes. Servicio de 04:30 a 22:00.

Línea 43 · _generar_datos_sinteticos()

```
perfil = (
```

Perfil diario con pico de manana (7:00) y de tarde (17:30)

Línea 51 · _generar_datos_sinteticos()

```
nivel = np.exp(np.cumsum(rng.normal(0, 0.03, len(dias))))
```

Nivel diario que deriva lentamente: hace utiles los rezagos

---

frecuencias/analitica/serie.py

28 líneas · 1 nota del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Convierte la demanda en una grilla regular de 15 minutos por ruta, en la que las franjas sin dato quedan como faltantes (antes en modelo_demanda.py).

Línea 26 · preparar_serie()

```
g["observado"] = g["subidas"].notna()
```

sin servicio o sin dato -> faltante, no cero

---

frecuencias/analitica/features.py

101 líneas · 15 notas del código original · 1 nota nueva

> **Descripción del archivo · nueva · 15 oct 2026**
> Construcción de las 22 variables del modelo (antes en modelo_demanda.py). La usan el entrenamiento, la API y el notebook de ingeniería de variables, de modo que todos calculan exactamente lo mismo.

Línea 15

```
"id_ruta", "minuto_dia", "dia_semana", "mes", "dia_del_anio",
```

calendario (se conoce de antemano)

Línea 17

```
"lag_h", "lag_h_mas_1", "media_1h_previa", "ratio_hoy", "nivel_28d",
```

demanda reciente (hasta la franja de referencia t - h)

Línea 18

```
"lag_1d", "lag_7d", "prom_4_semanas", "lag_364d",
```

estacionalidad semanal y anual

Línea 19

```
"viajes_1h_previa", "viajes_habituales",
```

oferta de viajes: la reciente y la habitual (la futura no se conoce, ver EDA)

Línea 23 · dias_en_rangos() · nota nueva · 15 oct 2026

```
def dias_en_rangos(rangos: pd.DataFrame) -> set:
```

Antes _dias_en_rangos; ahora es pública porque eda.ipynb también la usa.

Línea 33 · agregar_calendario()

```
d0, d1 = dia.min(), dia.max()
```

Solo lo que cae dentro de la serie: mismo resultado, mucho mas rapido en el servicio

---

Línea 46 · agregar_calendario()

```
tipo = np.select([df["dia_semana"] == 5, df["dia_semana"] == 6], [1, 2], 0)
```

0 = habil, 1 = sabado, 2 = domingo o festivo. El lunes y martes de Carnaval se comportan como festivo en Barranquilla, asi que tambien cuentan como tipo 2 (ver eda.ipynb, seccion 6).

Línea 51 · agregar_calendario()

```
df["impacto_evento"] = 0.0
```

Eventos (Guacherna, Batalla de Flores, partidos...): % de impacto en su ventana horaria

---

Línea 61 · calcular_features() · docstring · actualizada · 15 oct 2026

```
def calcular_features(serie: pd.DataFrame, horizonte_min: int, calendario) -> pd.DataFrame:
```

Variables de todas las franjas de la grilla, con o sin servicio. La usan el entrenamiento y el servicio FastAPI (frecuencias/api/main.py), asi ambos calculan exactamente igual. Para la franja t con horizonte h, lo observado solo se toma hasta t - h (la referencia): shift(h) o mas.

Línea 72 · calcular_features()

```
df["lag_h"] = g.shift(h)
```

ultimo valor conocido al predecir

Línea 75 · calcular_features()

```
df["lag_1d"] = g.shift(PASOS_DIA)
```

96 >= h para los tres horizontes

Línea 79 · calcular_features()

```
df["lag_364d"] = g.shift(PASOS_ANIO)
```

estacionalidad anual, mismo dia de la semana

Línea 81 · calcular_features()

```
df["nivel_28d"] = g.transform(
```

Nivel de la ruta: subidas medias por franja con servicio en los 28 dias previos a la referencia (sigue la caida de ~7% anual que muestra el EDA)

Línea 85 · calcular_features()

```
acum = df.assign(_s=df["subidas"].fillna(0)).groupby(["id_ruta", df["ts"].dt.normalize()])["_…
```

Como va el dia: subidas acumuladas del dia hasta la referencia frente a lo habitual a esa misma hora (promedio de las 4 semanas previas). Capta dias atipicos que el calendario no anuncia (lluvia, paros, eventos).

Línea 91 · calcular_features()

```
gv = df.groupby("id_ruta")["nro_viajes"]
```

Oferta: viajes con lecturas en la ultima hora conocida y los habituales en la franja objetivo. Los viajes programados de la franja objetivo NO se usan: se registran ~12 min antes de salir, no se conocen a 30-120 min.

Línea 100 · construir_features()

```
df = calcular_features(serie, horizonte_min, calendario)
```

Se entrena en toda franja con servicio (XGBoost tolera rezagos faltantes). Se evalua solo donde existen ambas lineas base.

---

frecuencias/inferencia/evaluacion.py

24 líneas · 1 nota del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Métricas de error (MAE, RMSE, WAPE y sMAPE), formato del reporte y clasificación de días normales y especiales (antes en modelo_demanda.py).

Línea 22 · tipo_periodo() · docstring

```
def tipo_periodo(df: pd.DataFrame) -> np.ndarray:
```

Dias especiales (festivo, Carnaval, Semana Santa) frente a dias normales.

---

frecuencias/inferencia/entrenamiento.py

137 líneas · 4 notas del código original · 3 notas nuevas

Descripción del archivo · docstring original de modelo_demanda.py · actualizada · 15 oct 2026

Fuente: las lecturas de los sensores de pasajeros (conteo_pasajeros_destino)
de 2023 a 2025, agregadas por ruta y franja de 15 min segun la hora real de
cada subida. Las descarga frecuencias/ingesta/extraccion.py a datos/demanda_15min.parquet.
(Antes se usaba timbradas_segmentos: solo cubre 8 semanas de 2023 y sus horas
estan corridas 2-3 h respecto de los sensores y los despachos; ver eda.ipynb.)

Modos de ejecucion:
- Con datos/demanda_15min.parquet: datos reales (corra antes python -m frecuencias.ingesta.extraccion).
- Sin el: datos sinteticos con la misma forma, para probar el pipeline.

Que hace:
1. Carga subidas por ruta y franja de 15 min y aplica las reglas de calidad del EDA: dias sin lecturas del sensor (el ultimo de cada mes) y franjas con lecturas imposibles quedan como faltantes, no como cero.
2. Arma una grilla regular de 15 min por ruta.
3. Para cada horizonte (30, 60, 120 min, RF-005) construye variables que solo usan informacion disponible al momento de predecir (seccion 9.3): calendario, demanda reciente, estacionalidad semanal y anual, nivel de la ruta y oferta de viajes. El detalle esta en feature_engineering.ipynb.
4. Compara XGBoost contra dos lineas base (seccion 9.1).
5. Reporta MAE, RMSE, WAPE y sMAPE, global, por ruta y por tipo de dia.
6. Calcula importancia de variables con SHAP (seccion 9.4).
7. Guarda cada modelo con sus metadatos en la carpeta modelos/ (seccion 9.2).

Los modelos guardados se sirven con frecuencias/api/main.py (FastAPI).

Pendiente (siguientes escalones): nivel segmento y sentido, intervalos de confianza.

Línea 20 · nuevo_modelo() · nota nueva · 11 oct 2026

```
def nuevo_modelo(n_estimators=3000, early_stopping_rounds=50):
```

Tope alto de árboles (3000) para que sea la validación la que decida cuántos usar: el early stopping detiene el entrenamiento tras 50 rondas sin mejora. El modelo final se crea con el número de árboles ya elegido y sin early stopping.

Línea 24 · nuevo_modelo()

```
objective="count:poisson",
```

la demanda es un conteo >= 0

Línea 30 · nuevo_modelo()

```
early_stopping_rounds=early_stopping_rounds,
```

usa val; test queda intacto

Línea 36 · particion_temporal() · docstring

```
def particion_temporal(df: pd.DataFrame, frac_val=0.1, frac_test=0.2):
```

Cortes cronologicos calculados sobre el rango con lineas base.

Línea 46 · main() · nota nueva · 15 oct 2026

```
def main():
```

Antes era el bloque if __name__ == "__main__" de modelo_demanda.py; ahora es una función y se ejecuta con python -m frecuencias.inferencia.entrenamiento.

Línea 104 · main() · nota nueva · 11 oct 2026

```
n_arboles = modelo.best_iteration + 1
```

Modelo final: con el número de árboles que eligió la validación, se reentrena con todas las franjas con servicio (train + val + test, hasta el último dato) y ese es el modelo que se guarda y sirve la API. Las métricas de prueba que se imprimen y se guardan en los metadatos son las del modelo evaluado antes de este paso, porque el modelo final ya vio el periodo de prueba.

Línea 110 · main()

```
base = CARPETA_MODELOS / f"demanda_h{horizonte}"
```

Artefacto + metadatos (seccion 9.2)

---

frecuencias/inferencia/modelos.py

32 líneas · 1 nota del código original · 1 nota nueva

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Carga los modelos guardados en modelos/ y verifica que se entrenaron con las mismas variables que usa el código (antes _cargar_modelos en api_demanda.py).

Línea 8 · nota nueva · 15 oct 2026

```
COMANDO_ENTRENAMIENTO = "python -m frecuencias.inferencia.entrenamiento"
```

Comando que muestran los mensajes de error cuando hay que reentrenar.

Línea 22 · cargar_modelos()

```
if meta["feature_version"] != FEATURE_VERSION or meta["features"] != FEATURES:
```

Un modelo entrenado con otras variables predeciria basura en silencio

---

frecuencias/api/esquemas.py

37 líneas · 0 notas del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Esquemas Pydantic de las respuestas de la API (antes en api_demanda.py).

frecuencias/api/main.py

188 líneas · 6 notas del código original · 4 notas nuevas

Descripción del archivo · docstring original de api_demanda.py · actualizada · 15 oct 2026

Servicio FastAPI de prediccion de demanda - SOBUSA Data Challenge 2026.

Sirve los modelos que entrena frecuencias/inferencia/entrenamiento.py (carpeta modelos/) y calcula
las variables con las mismas funciones del entrenamiento, para que el modelo
reciba al predecir exactamente lo que vio al entrenar.

Al arrancar carga en memoria la copia local de datos/ que genera
frecuencias/ingesta/extraccion.py (subidas por ruta y franja de 15 min de 2023 a 2025, ya
limpias con las mismas reglas del entrenamiento), el calendario y los nombres
de las rutas. Cada prediccion se resuelve en memoria, sin consultar la base.
Para tomar datos nuevos, corra python -m frecuencias.ingesta.extraccion y reinicie el servicio.

Ejecucion:
    uvicorn frecuencias.api.main:app --reload (o: python -m frecuencias.api.main)
Documentacion interactiva: http://127.0.0.1:8000/docs

Línea 18 · nota nueva · 15 oct 2026

```
PREFIJO = "/api/v1"
```

Ruta base /api/v1 que exige la especificación (§12.1). Los endpoints quedan en /api/v1/salud, /api/v1/modelos, /api/v1/rutas y /api/v1/prediccion/{id_ruta}; la raíz / redirige a /docs.

Línea 19

```
ZONA_HORARIA = "America/Bogota"
```

las franjas de la base estan en hora local

Línea 20

```
HORAS_MINIMAS_HISTORIA = 48
```

sin datos en las 48 h previas no se predice (cubre el dia sin sensor de fin de mes)

Línea 21

```
DIAS_HISTORIA = 371
```

lag_364d mira 364 dias atras, con margen

Línea 36 · lifespan() · nota nueva · 15 oct 2026

```
estado["motor_ia"] = db.motor_destino() if db.url_destino() else None
```

Conexión a la base ai_frecuencias si está configurada; se libera al apagar el servicio.

Línea 67 · _serie_conocida() · docstring

```
def _serie_conocida(ruta: pd.DataFrame, referencia: pd.Timestamp, hasta: pd.Timestamp) -> pd.…
```

Grilla regular de la ruta desde DIAS_HISTORIA dias antes de la referencia hasta la ultima franja objetivo. Lo posterior a la referencia se borra: al predecir solo se conoce lo ocurrido hasta la referencia.

Línea 77 · _estado_base_ia() · nota nueva · 15 oct 2026

```
def _estado_base_ia() -> dict:
```

Informa en /api/v1/salud si la base de IA no está configurada, si responde (con la revisión de Alembic aplicada) o si falla.

Línea 105 · modelos() · docstring

```
def modelos():
```

Metadatos de cada modelo (seccion 9.2), con sus metricas de prueba.

Línea 125 · prediccion() · docstring

```
def prediccion(
```

Subidas previstas en la ruta a 30, 60 y 120 minutos de la referencia.

Línea 182 · nota nueva · 15 oct 2026

```
app.include_router(router)
```

El router se registra al final, cuando ya tiene todos sus endpoints.

Migraciones de la base ai_frecuencias

migrations/env.py

24 líneas · 0 notas del código original · 2 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Entorno de Alembic. Toma la conexión de frecuencias/db.py (variables AI_DB_*), crea el esquema ai_frecuencias si no existe y guarda dentro de ese esquema la tabla de versiones de Alembic, de modo que nada se escribe en el esquema public. En modo offline (alembic upgrade head --sql) genera el DDL sin conectarse a la base.

Línea 16 · _migrar() · nota nueva · 15 oct 2026

```
context.execute(CreateSchema(ESQUEMA_IA, if_not_exists=True))
```

El esquema se crea en la misma transacción y antes que la tabla de versiones de Alembic, que vive dentro de él.

Línea 20 · nota nueva · 15 oct 2026

```
if context.is_offline_mode():
```

Con --sql no se conecta: escribe el DDL completo para revisarlo o entregarlo (E04).

migrations/versions/0001_esquema_inicial.py

294 líneas · 0 notas del código original · 8 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Migración inicial: crea las siete tablas del esquema ai_frecuencias que pide la especificación (§8.1 y §8.2), con sus restricciones e índices, y carga como datos semilla los umbrales sugeridos en el §5.1. El downgrade borra las tablas en orden inverso.

Línea 25 · nota nueva · 15 oct 2026

```
PARAMETROS_INICIALES = [
```

Valores iniciales sugeridos en el §5.1. Quedan marcados como no validados (validado = false) hasta que el área operativa los confirme; headway_minimo queda sin valor porque lo define la operación.

Línea 49 · _crear_parametros() · nota nueva · 15 oct 2026

```
def _crear_parametros() -> None:
```

Tabla operational_parameter: umbrales configurables y versionados (regla de gobierno del §5.1). Cada cambio es una fila nueva con version + 1 y solo puede haber una versión vigente por clave y ruta (índice único parcial uq_parametro_vigente). id_ruta nulo significa valor general.

Línea 84 · _crear_modelos_y_ejecuciones() · nota nueva · 15 oct 2026

```
"model_registry",
```

Registro de modelos (§9.2, RF-018 y RF-019): artefacto con su SHA-256, variables, hiperparámetros, datos usados, métricas y estado (candidato, activo, retirado o rechazado). Solo puede haber un modelo activo por horizonte.

Línea 117 · _crear_modelos_y_ejecuciones() · nota nueva · 15 oct 2026

```
"execution_log",
```

Bitácora de ejecuciones (RF-013, §11.2): tipo, estado, marca de agua leída, registros leídos y rechazados, y un correlation_id para la trazabilidad.

Línea 146 · _crear_predicciones_y_recomendaciones() · nota nueva · 15 oct 2026

```
"prediction",
```

Predicciones (§9.2, RF-006): valor puntual, límites del intervalo, versión de variables, data_cutoff, latencia y explicación. Una sola fila por modelo, ruta, sentido, segmento, franja y horizonte, de modo que una carga repetida no duplica (§14.2).

Línea 187 · _crear_predicciones_y_recomendaciones() · nota nueva · 15 oct 2026

```
"recommendation",
```

Recomendaciones (RF-007): headway, buses por hora y vehículos requeridos. Si la recomendación no es viable, el motivo es obligatorio (§10.3).

Línea 219 · _crear_alertas() · nota nueva · 15 oct 2026

```
"alert",
```

Alertas (RF-010, §5.3): código ALT-001 a ALT-010, severidad, estado, valores observado y esperado, umbral, explicación y recomendación. No puede haber dos alertas abiertas con la misma clave de deduplicación (§14.2).

Línea 267 · _crear_alertas() · nota nueva · 15 oct 2026

```
"alert_history",
```

Bitácora de cambios de estado y comentarios de cada alerta (RF-011); se borra en cascada junto con la alerta.

---

migrations/script.py.mako

15 líneas · 0 notas del código original · 0 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Plantilla con la que Alembic genera nuevas migraciones (python -m alembic revision -m "descripcion"). Es la plantilla estándar sin comentarios ni docstrings.

alembic.ini

38 líneas · 0 notas del código original · 1 nota nueva

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Configuración de Alembic: carpeta de migraciones y registro en consola. La cadena de conexión no está aquí; la arma migrations/env.py a partir de las variables AI_DB_*.

Línea 3 · nota nueva · 15 oct 2026

```
prepend_sys_path = .
```

Permite importar el paquete frecuencias al ejecutar alembic desde la raíz del proyecto.

Contenedores, configuración y dependencias

Dockerfile

28 líneas · 0 notas del código original · 3 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Imagen de la API: Python 3.12 slim (la especificación exige 3.11 o superior), dependencias fijadas y un usuario sin privilegios. Al arrancar aplica las migraciones pendientes y levanta uvicorn en el puerto 8000. Los datos y los modelos no se copian a la imagen: se montan desde el equipo.

Línea 8 · nota nueva · 15 oct 2026

```
RUN apt-get update \
```

Instala libgomp1, la biblioteca OpenMP que XGBoost necesita en Linux.

Línea 21 · nota nueva · 15 oct 2026

```
RUN useradd --create-home --uid 1000 app \
```

La API corre con un usuario sin privilegios (uid 1000), no como root.

Línea 28 · nota nueva · 15 oct 2026

```
CMD ["sh", "-c", "alembic upgrade head && exec uvicorn frecuencias.api.main:app --host 0.0.0.…
```

Aplica las migraciones pendientes y luego arranca la API.

docker-compose.yml

42 líneas · 0 notas del código original · 5 notas nuevas

> **Descripción del archivo · nueva · 15 oct 2026**
>
> Levanta la solución completa (RNF-008): el servicio db (PostgreSQL 14 con PostGIS 3.5, datos persistentes en el volumen datos_ai) y el servicio api (la imagen de este repositorio), que espera a que la base esté sana antes de arrancar.

Línea 5 · nota nueva · 15 oct 2026

```
image: postgis/postgis:14-3.5
```

PostgreSQL 14 con PostGIS, la versión que exige la especificación (§7.1 y §8.1).

Línea 9 · nota nueva · 15 oct 2026

```
POSTGRES_PASSWORD: ${AI_DB_PASSWORD:?Defina AI_DB_PASSWORD en .env}
```

Si AI_DB_PASSWORD no está en .env, docker compose se detiene con ese mensaje.

Línea 13 · nota nueva · 15 oct 2026

```
- "${AI_DB_PORT:-5433}:5432"
```

La base queda expuesta en el puerto 5433 del equipo para no chocar con un PostgreSQL local.

Línea 26 · nota nueva · 15 oct 2026

```
AI_DB_HOST: db
```

Dentro de la red de Docker la base se llama db y escucha en el puerto 5432. La API recibe solo las variables de ai_frecuencias: no necesita credenciales de la base del reto.

Línea 37 · nota nueva · 15 oct 2026

```
- ./datos:/app/datos:ro
```

datos/ y modelos/ se montan en solo lectura desde el equipo.

.env.example

13 líneas · 0 notas del código original · 0 notas nuevas

Descripción del archivo · nueva · 15 oct 2026

Plantilla de configuración sin secretos ni datos de conexión (§7.1 y §19): se copia a .env y se completa con los valores que entrega Extreme Technologies. La contraseña de la base del reto se pasa en la variable PGPASSWORD o SOBUSA_DB_PASSWORD, nunca en archivos del repositorio.

.gitignore

4 líneas · 0 notas del código original · 0 notas nuevas

Descripción del archivo · nueva · 15 oct 2026

Excluye de git el archivo .env, las cachés de Python y los puntos de control de Jupyter.

.dockerignore

16 líneas · 0 notas del código original · 0 notas nuevas

Descripción del archivo · nueva · 15 oct 2026

Deja fuera de la imagen el historial de git, los secretos, las cachés, los notebooks, los PDF, los datos y los modelos.

requirements.txt

12 líneas · 0 notas del código original · 0 notas nuevas

Descripción del archivo · nueva · 15 oct 2026

Dependencias de ejecución, fijadas a las versiones con las que se probó el proyecto (Python 3.12). alembic es nueva.

requirements-dev.txt

3 líneas · 0 notas del código original · 0 notas nuevas

Descripción del archivo · nueva · 15 oct 2026

Dependencias que solo usan los notebooks (matplotlib e ipykernel), además de las de ejecución.